Journal of Statistical Software

MMMMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

Supplementary Benchmark Results for fastEmbedR

Moussa Kassim Mohamed[1,2,*] **Martin Ocharo**[1,2,*]
Giulia Canarutto[3,4] Dalia Ahmed[1] Dupe Ojo[1]
Alessia Vignoli[5,6] Leonardo Tenori[5,6,†]
Dinesh Gupta[7] Silvano Piazza[3,8]
Stefano Cacciatore[1,2,†]

Abstract

This supplement records the completed observations and explicit failures of diagnostic campaign 20260930T203319Z: workflow timing and memory, embedding quality, neighbor recall, support width, CPU scaling, held-out transformation, landmarking, PCA, and graph clustering. A separate September 29 campaign supplies the illustrative embedding gallery. Neither campaign passed its final audit; results are not release-level speed claims. The experimental Deep1B file-backed protocol is specified separately, without treating unfinished runs as results.

Keywords: dimensionality reduction, t-SNE, UMAP, reproducible benchmarks.

1. Workflow elapsed times

Table S1 combines the R and direct-Python results for PCA, t-SNE, and UMAP without treating their timing boundaries as equivalent. It gives every observed completed time, marks single-fit measurements, and identifies failed or missing combinations. The source campaign used full data-set matrices and failed its final audit; the times are descriptive diagnostics, not publication-grade speed ratios. "R call" is the complete public fit; "Py fit" is the direct Python estimator fit, excluding process startup. An asterisk marks a completed single fit without timing repetitions. T denotes a time limit, M a confirmed memory limit, F another failure, K an unclassified kill, U unsupported, C cancelled, I a completed fit without usable time, and – not run or no status.

Table S1: Observed elapsed time in seconds from campaign 20260930T203319Z (diagnostic, failed audit). R call and direct Python fit use different timing boundaries. All fits use the full source matrix.

PCA, t-SNE, and UMAP methods

A. PCA, data sets 1/2

Method	Scope	COIL-20	USPS	Fashion	FlowRepo.	flow18	MNIST
fastEmbedR [CPU]	R call	2.89	0.26	7.64	19.9	0.63	7.52
irlba [CPU]	R call	0.89	0.099	1.78	455	0.75	1.77
fastEmbedR [CUDA]	R call	0.020	0.006	0.039	1.56	0.042	0.043
scikit-learn PCA [Python]	Py fit	0.45	0.050	0.96	19.0	1.10	1.03
cuML [CUDA]	Py fit	6.05	0.027	0.33	2.68	0.070	0.34

A. PCA, data sets 2/2

Method	Scope	ImageNet	MetRef	mass41	Tabula	Retina
fastEmbedR [CPU]	R call	230	0.035	0.72	0.81	0.38
irlba [CPU]	R call	101	0.026	38.7	0.61	0.46
fastEmbedR [CUDA]	R call	0.89	0.005	0.054	0.017	0.010
scikit-learn PCA [Python]	Py fit	16.0	0.006	0.68	0.31	0.086
cuML [CUDA]	Py fit	19.7	0.005	0.10	0.044	0.021

B. t-SNE, data sets 1/2

Method	Scope	COIL-20	USPS	Fashion	FlowRepo.	flow18	MNIST
Rtsne [CPU]	R call	12.0	15.4	426	T	1726*	825
FIt-SNE [CPU]	R call	15.0	6.64	84.1	3702*	736	109
fastEmbedR [CPU]	R call	18.8	11.9	125	3354*	300	129
fastEmbedR [CUDA]	R call	0.86	0.22	1.39	F	6.07	1.40
scikit-learn t-SNE [Python]	Py fit	5.19	66.8	711	T	T	774
openTSNE [Python]	Py fit	40.9	108	242	T	1098*	241
cuML [CUDA]	Py fit	5.94	0.41	2.42	F	51.4	2.39

B. t-SNE, data sets 2/2

Method	Scope	ImageNet	MetRef	mass41	Tabula	Retina
Rtsne [CPU]	R call	T	0.86	2746*	157	70.1
FIt-SNE [CPU]	R call	2541*	3.81	390	105	59.1
fastEmbedR [CPU]	R call	1672*	9.80	300	86.5	27.7
fastEmbedR [CUDA]	R call	505	0.16	6.29	3.24	0.42
scikit-learn t-SNE [Python]	Py fit	T	1.54	T	648	226
openTSNE [Python]	Py fit	1484*	11.8	584	125	97.0
cuML [CUDA]	Py fit	183	0.19	48.7	2.63	1.27

C. UMAP, data sets 1/2

Method	Scope	COIL-20	USPS	Fashion	FlowRepo.	flow18	MNIST
umap [CPU]	R call	181	145	2704*	T	T	2223*
uwot [CPU]	R call	14.6	7.12	62.8	6016*	1302*	65.9
uwot fast SGD [CPU]	R call	13.7	4.56	51.4	3958*	729	52.6
fastEmbedR fuzzy [CPU]	R call	6.77	3.29	79.7	2987*	319	86.2
fastEmbedR binary [CPU]	R call	6.99	7.60	127	5212*	762	113
fastEmbedR fuzzy [CUDA]	R call	0.77	0.12	1.03	F	1.14	1.03
fastEmbedR binary [CUDA]	R call	0.80	0.12	1.09	F	1.69	1.10
umap-learn [Python]	Py fit	11.6	8.36	85.5	T	1636*	52.1
cuML [CUDA]	Py fit	0.18	0.14	0.87	F	9.11	0.87

Runtime table, continued

C. UMAP, data sets 2/2

Method	Scope	ImageNet	MetRef	mass41	Tabula	Retina
umap [CPU]	R call	T	4.13	T	1806*	469
uwot [CPU]	R call	2643*	1.36	590	106	25.5
uwot fast SGD [CPU]	R call	1710*	0.80	485	61.9	13.5
fastEmbedR fuzzy [CPU]	R call	1559*	0.22	307	26.5	17.6
fastEmbedR binary [CPU]	R call	2320*	0.44	756	75.8	34.3
fastEmbedR fuzzy [CUDA]	R call	506	0.086	1.51	0.35	0.14
fastEmbedR binary [CUDA]	R call	497	0.088	2.10	0.44	0.19
umap-learn [Python]	Py fit	5654*	1.48	885	64.3	28.1
cuML [CUDA]	Py fit	307	0.18	6.51	0.72	0.30

The following figures inventory the failed-audit diagnostic campaign. They are not the frozen-source figures required for a runtime claim.

Table S2: Reserved frozen-source workflow comparison. XXX denotes a result not yet verified, not zero or a completed fit. R public-call and direct-Python fit boundaries remain distinct; their times must not form a ratio. Quality differences require the same fixed sample. NOMAD has a distinct objective and is descriptive only.

Family	Comparator and scope	Pairs	fastEmbedR (s)	Comparator (s)	Quality difference
PCA	irlba, R call	XXX	XXX	XXX	XXX
PCA	scikit-learn, Python fit	XXX	XXX	XXX	XXX
PCA	cuML, Python fit	XXX	XXX	XXX	XXX
t-SNE	Rtsne, R call	XXX	XXX	XXX	XXX
t-SNE	FIt-SNE, process wall	XXX	XXX	XXX	XXX
t-SNE	openTSNE, Python fit	XXX	XXX	XXX	XXX
t-SNE	scikit-learn, Python fit	XXX	XXX	XXX	XXX
t-SNE	cuML, Python fit	XXX	XXX	XXX	XXX
UMAP	uwot, R call	XXX	XXX	XXX	XXX
UMAP	umap, R call	XXX	XXX	XXX	XXX
UMAP	umap-learn, Python fit	XXX	XXX	XXX	XXX
UMAP	cuML, Python fit	XXX	XXX	XXX	XXX
Other	NOMAD, CUDA fit	XXX	–	XXX	XXX

The replacement PCA, t-SNE, and UMAP figures must use the same frozen package image, method manifest, complete-case inventory, and timing boundaries as this table. Failed cases remain in the denominator, rather than disappearing when a median of successful cases is computed. The quality-difference column will name its metric and direction when filled.

A separate candidate-image paired CUDA run fitted all 11 complete source data sets for each of six requested methods (66 full-row layouts). Its benchmark manifest and archived source files passed SHA-256 verification. The audit reported one warning: cuML t-SNE on FlowRepository_FR-FCM-ZYRM_files returned zero native fitted KL, whereas its independent 2,000-row sampled KL was 2.2245 and trustworthiness was 0.9471. This supports a nondegenerate embedding but does not recover the full fitted objective. cuML 26.06 floors normalized probabilities at `FLT_EPSILON` in its FFT KL kernel; the typical normalized edge weight for 5.22 million rows and 90 neighbors is below that floor. Numerical flooring is a plausible cause, not a proven diagnosis for every edge. We retain the native zero and exclude its 853.9-second median fit time from ratios. We reported the observation and a kernel-level

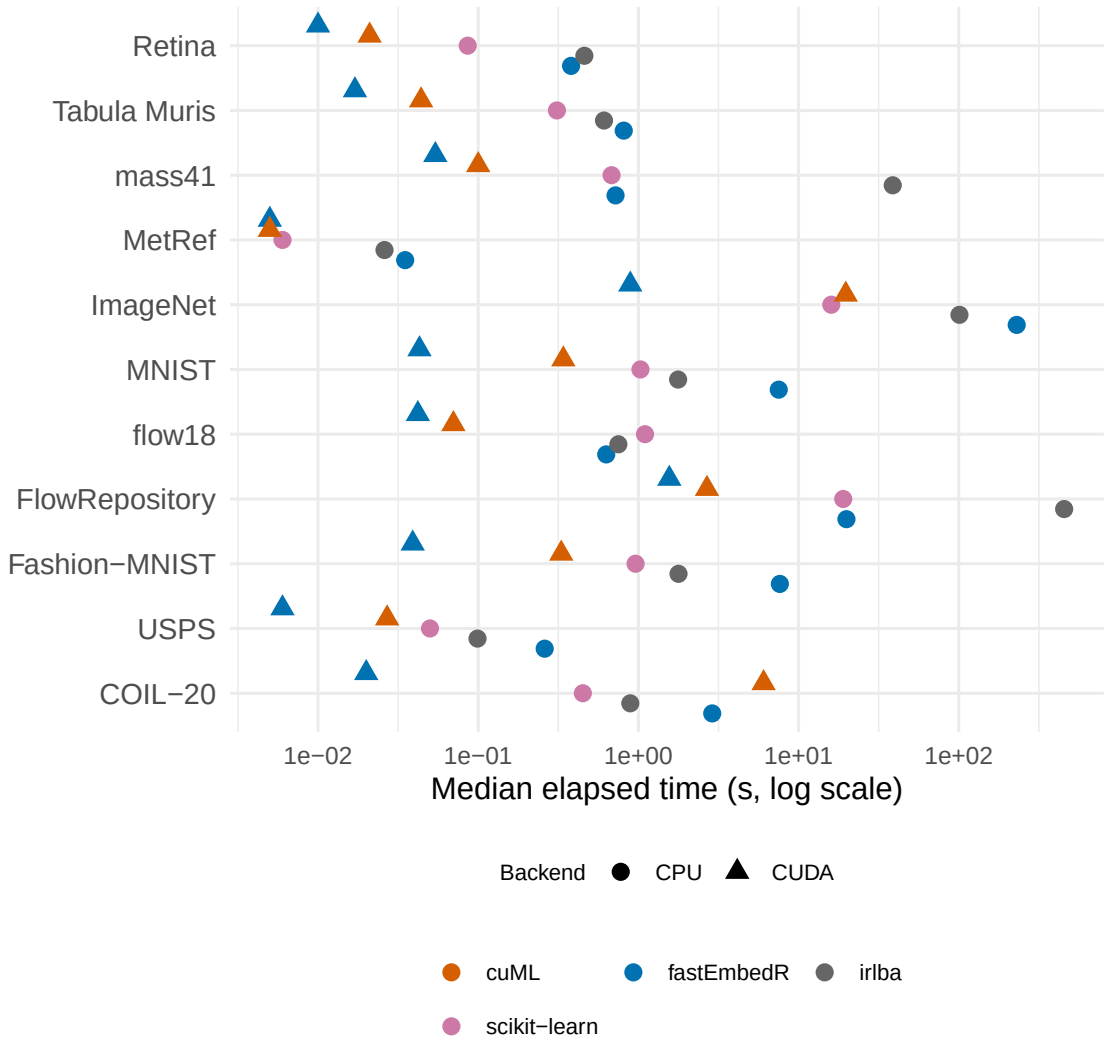

Figure S1: Observed PCA elapsed times in diagnostic campaign 20260930T203319Z. Circles denote CPU, triangles CUDA, and fastEmbedR retains one color across all three figures. Points show archived medians without quartile intervals. R-call and Python-fit boundaries, single fits, and failures are identified in the workflow table.

counterexample in NVIDIA/cuml issue 8795; no corrected cuML release has been verified for this benchmark. The paired campaign therefore has 65 timing-eligible method–data-set results. Table S3 reports descriptive differences on the shared post-fit quality sample. It is not a cross-language speed comparison, and it does not complete the separate extended JSS or Deep1B audits.

The extended JSS run subsequently ended without a final passing audit. Its archived failure ledger records a failed CUDA-pair worker, a cancelled CUDA quality task, a failed CUDA controller, and a cancelled long CPU quality task. These cases remain in the requested denominator; completed CPU or CUDA files do not constitute a paired comparison. The Deep1B matrix ledgers likewise contain both passing cases and failed or missing case audits.

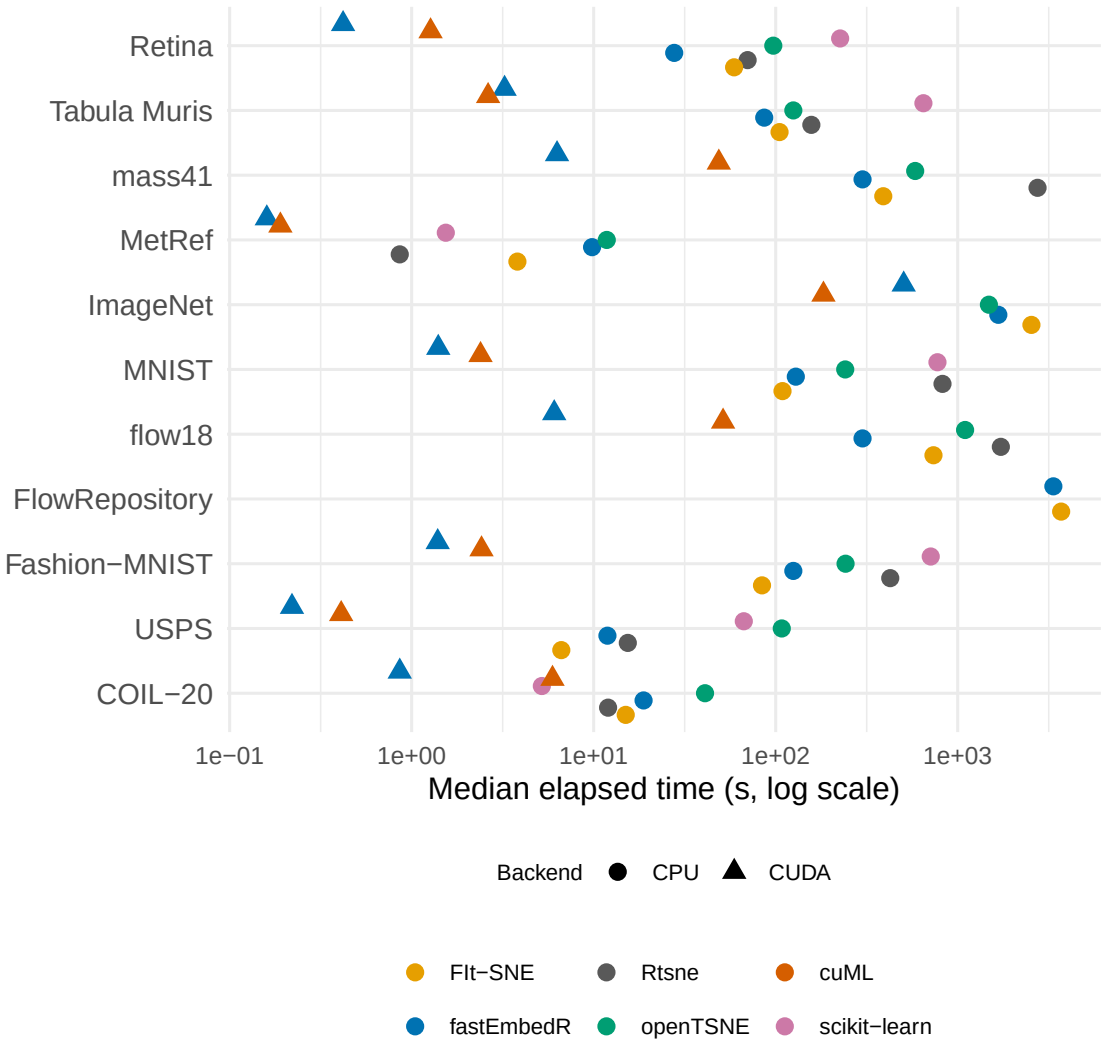

Figure S2: Diagnostic two-dimensional t-SNE medians. Circles denote CPU and triangles CUDA. Public R calls and direct Python fits retain different timing boundaries; single fits and missing cases remain in the workflow table.

Until a source-matched, complete comparison passes, the reserved timing and scaling cells below remain unfilled. UCT HPC GPU submission is suspended pending user and administrator approval.

NOMAD also completed 11 full-row layouts in that run but has a distinct objective, so it is omitted from these within-family differences. The candidate container SHA-256 begins `91387e35888b`; the full image, source archive, DLL, and benchmark hashes remain in its campaign manifest.

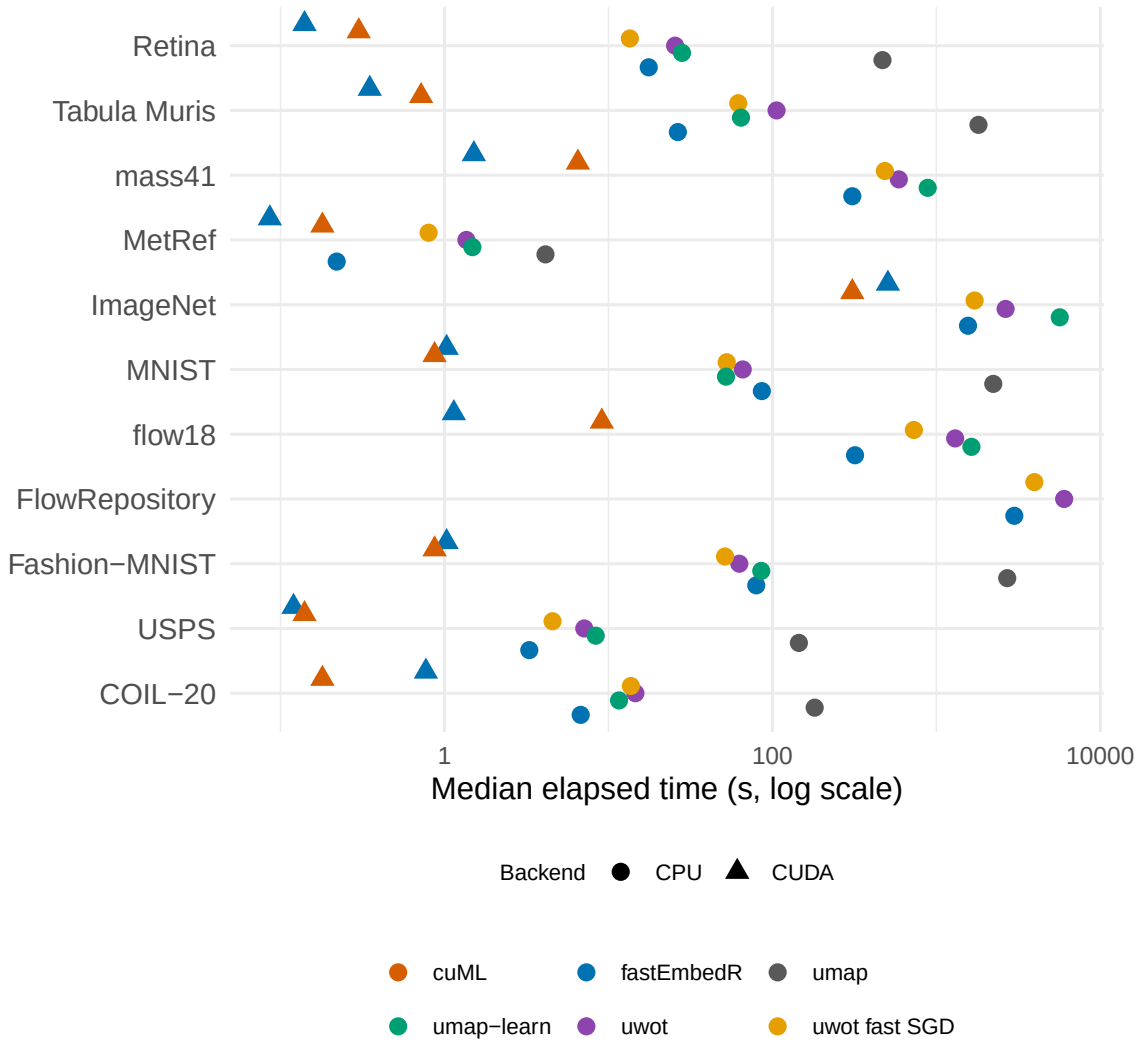

Figure S3: Diagnostic two-dimensional UMAP medians, excluding the secondary binary route. Circles denote CPU and triangles CUDA. Binary UMAP is retained in the sensitivity table; mixed timing boundaries and incomplete cases preclude a release-level speed ratio.

2. Comparator grid and timing boundaries

In the revised two-dimensional t-SNE protocol, **fastEmbedR** reports its selected grid in each fit. The pinned FIt-SNE wrapper exposes interpolation points, minimum intervals, and intervals per coordinate unit. The benchmark requests three interpolation points and minimum interval counts of 43 below 40,000 rows and 86 otherwise. FIt-SNE rounds these to supported box counts of at least 50 and 90, respectively, corresponding to at least 150 and 270 interpolation nodes per axis. The realized grid can be larger if the current coordinate span requires it; the wrapper does not return that size. These settings approximate rather than exactly match the **fastEmbedR** CPU 128- and 256-cell policies. FIt-SNE three-dimensional Barnes–Hut runs do not use the interpolation grid.

Table S3: Candidate-image paired CUDA quality differences, fastEmbedR minus cuML, with data set as the unit of aggregation. Entries are the median of 11 within-data-set differences, not the difference of two pooled medians. The cuML t-SNE quality sample remains scored for the invalid-KL data set, but its timing is excluded. These within-sample scores do not establish full-reference neighborhood preservation.

Pair	Quality pairs	Timing-eligible pairs	Median Δ trust / Preserve@30
t-SNE / cuML t-SNE	11	10	$+0.01546 / +0.01223$
Fuzzy UMAP / cuML UMAP	11	11	$-0.00346 / -0.00055$
Binary UMAP / cuML UMAP	11	11	$+0.00383 / +0.00800$

The tested cuML 26.6.0 implementation fixes three interpolation points and 50 intervals per axis in its two-dimensional FFT source, yielding a 300-cell padded transform axis. Its public Python t-SNE constructor does not expose a grid-resolution parameter, so a matched-grid cuML comparison would require modifying and rebuilding cuML. The benchmark records this fixed source policy, but does not mislabel the runs as grid matched. UMAP methods do not use this t-SNE FFT grid.

The September 30 workflow campaign has five same-seed timing repetitions after warm-up for timing-eligible methods; jobs reaching the long-run budget may have one completed fit. All input matrices were fitted in full. The 7,200-second per-method limit and failed campaign audit are reflected in Table S1. The embedding gallery below comes from a separate, capped September 29 campaign and must not be read as visual evidence for the September 30 timings.

3. Approximate-neighbor calibration

The reported CPU HNSW workflows used a fixed, shape- and k-based policy, not a search over parameters fitted to each evaluation data set. The requested 0.99 recall is therefore an objective rather than a measured guarantee. Exact-reference queries in the campaign were used for auditing after settings had been chosen; their cost is diagnostic, not a hidden component of the fixed-policy fit time. A result below 0.99 remains a recall miss regardless of its runtime.

A data-adaptive comparison requires a separate arm. Candidate index-build settings (M and `efConstruction`) must be distinguished from the runtime search setting `efSearch`. Candidate selection may use exact neighbors of a seeded tuning-query sample, but not the disjoint holdout queries used to report final recall. The selected settings and their selection rule must be frozen before the holdout and downstream UMAP or t-SNE quality evaluations. End-to-end adaptive time includes exact-reference generation, candidate construction and search, and the final index and embedding fit. Search-only time and projections amortized over repeated queries are reported separately, never substituted for that total. The calibration budget, query IDs, source identity, build/search parameters, thread count, and observed recall distribution must accompany each result. An untouched query set checks selection within a data set, whereas transfer to new data sets checks whether the policy itself generalizes. A data set used to design the candidate grid is not an independent test of that grid. No adaptive-policy speed or recall result is inferred from the fixed-policy campaign in this supplement.

4. Data-set-level CUDA embedding quality

Table S4 reports trustworthiness, Preserve@30, and sampled t-SNE KL for every CUDA **fastEmbedR** and cuML pair. The methods use the same fitted row count and evaluation sample within a data set, but differ in affinity support and timing boundary. KL values across those objectives are not directly comparable.

5. Backend quality and t-SNE support

Table S5 reports full-workflow quality for every completed CPU and CUDA data-set pair. The CPU FlowRepository full-workflow quality run was incomplete; matched-neighbor quality from a different boundary is not substituted. Trustworthiness and Preserve@30 use fixed quality-sample rows. The low ImageNet values reflect this feature matrix's limited two-dimensional neighborhood preservation; ImageNet is used primarily as a scalability stress test. Final KL was not retained for the CUDA full-workflow diagnostic and is shown as missing, not zero.

The $k = P$ affinity support makes the target perplexity the maximum possible entropy of each conditional distribution. At $P = 30$, normalized conditional entropy was approximately 1.000 for $k = P$ and 0.761 for $k = 3P$ in an exact-neighbor affinity diagnostic. Table S6 gives paired time and quality summaries for $P = 10$, 20, and 30. KL values across support widths are objectives with different affinities, not comparable optimization losses for one common objective.

6. Scaling, transformation, and landmarks

The strong-scaling experiment used identical inputs at 1, 2, 4, 8, and 12 CPU threads on COIL20, MNIST, flow18, and ImageNet. Times are complete workflows; stage-level values, including search, initialization, graph or affinity construction, and optimization, remain in the machine-readable campaign. The ratios in Table S7 are relative to each data set's one-thread result.

Held-out transformation fits a reference on training rows and projects genuinely unseen queries without moving the reference. Table S8 contrasts query-to-reference Preserve@30 with joint embedding of those queries. Times are transform calls, not complete fit-plus-transform workflows.

The 20-percent landmark experiment instead reconstructs rows omitted from the fitted subset. It is a different statistical task from held-out prediction. In Table S9, elapsed and host-RSS ratios are landmark/full; ratios below one favor landmarking. The GPU peak ratio is unavailable where device sampling was inadequate. In particular, the CUDA UMAP median runtime and host-RSS ratios exceed one. Procrustes agreement is especially modest for fuzzy UMAP.

7. Secondary analyses and memory

Binary UMAP discards fuzzy edge weights and therefore changes the embedding objective. It is retained as an adjacency sensitivity analysis. Table S10 reports its observed time and Preserve@30 change, without treating it as a faster fuzzy-UMAP setting.

Table S4: CUDA embedding quality in diagnostic campaign 20260930T203319Z. Entries are fastEmbedR / cuML on fixed sampled rows; n is the fitted matrix size. Status is fastEmbedR / cuML; – marks unavailable metrics. Different affinity supports make sampled t-SNE KL descriptive, not a matched-objective test.

A. Compact-support t-SNE / cuML t-SNE

Data set	Fitted n	Trustworthiness	Preserve@30	Sampled KL	Status
COIL-20	1,440	0.939 / 0.934	0.627 / 0.623	1.480 / 1.317	success / success
USPS	11,000	0.952 / 0.940	0.468 / 0.462	2.024 / 2.006	success / success
Fashion	70,000	0.973 / 0.966	0.485 / 0.491	1.877 / 1.744	success / success
FlowRepo.	5,220,347	– / –	– / –	– / –	failed / failed
flow18	1,000,021	0.967 / 0.951	0.445 / 0.422	2.049 / 1.857	success / success
MNIST	70,000	0.941 / 0.931	0.437 / 0.443	2.316 / 2.164	success / success
ImageNet	1,281,167	0.667 / 0.635	0.167 / 0.124	3.510 / 3.310	success / success
MetRef	873	0.848 / 0.797	0.442 / 0.358	1.846 / 2.153	success / success
mass41	965,282	0.968 / 0.949	0.407 / 0.396	2.128 / 1.960	success / success
Tabula	100,102	0.962 / 0.940	0.632 / 0.592	1.736 / 1.596	success / success
Retina	44,808	0.912 / 0.900	0.432 / 0.428	1.970 / 1.916	success / success

B. Fuzzy UMAP / cuML UMAP

Data set	Fitted n	Trustworthiness	Preserve@30	Status
COIL-20	1,440	0.908 / 0.920	0.613 / 0.620	success / success
USPS	11,000	0.885 / 0.883	0.358 / 0.363	success / success
Fashion	70,000	0.962 / 0.967	0.463 / 0.463	success / success
FlowRepo.	5,220,347	0.960 / 0.967	0.445 / 0.443	success / success
flow18	1,000,021	0.962 / 0.961	0.435 / 0.432	success / success
MNIST	70,000	0.939 / 0.935	0.452 / 0.444	success / success
ImageNet	1,281,167	0.585 / 0.647	0.104 / 0.161	success / success
MetRef	873	0.836 / 0.821	0.428 / 0.411	success / success
mass41	965,282	0.968 / 0.967	0.404 / 0.407	success / success
Tabula	100,102	0.923 / 0.970	0.607 / 0.643	success / success
Retina	44,808	0.900 / 0.904	0.406 / 0.411	success / success

Table S5: File-to-layout quality from campaign 20260930T203319Z. Each entry is CPU / CUDA, over successful full-workflow seeds. A missing CPU FlowRepository result is shown as –. KL is the production t-SNE objective; UMAP has no KL entry. The campaign audit failed.

Data set	Trustworthiness	Preserve@30	Final KL
Compact-support t-SNE			
COIL-20	0.938 / 0.940	0.630 / 0.627	0.794 / –
Fashion-MNIST	0.972 / 0.973	0.483 / 0.484	2.697 / –
flow18	0.960 / 0.966	0.438 / 0.444	2.887 / –
ImageNet	0.646 / 0.658	0.155 / 0.158	2.287 / –
Retina	0.906 / 0.912	0.424 / 0.431	2.941 / –
mass41	0.963 / 0.968	0.401 / 0.408	2.905 / –
MetRef	0.848 / 0.847	0.444 / 0.442	1.850 / –
MNIST	0.939 / 0.941	0.436 / 0.437	2.903 / –
Tabula Muris	0.949 / 0.960	0.619 / 0.634	2.183 / –
USPS	0.949 / 0.951	0.467 / 0.467	2.167 / –
FlowRepository	– / 0.964	– / 0.440	– / –
Fuzzy UMAP			
COIL-20	0.915 / 0.908	0.612 / 0.612	– / –
Fashion-MNIST	0.971 / 0.961	0.480 / 0.465	– / –
flow18	0.960 / 0.961	0.443 / 0.436	– / –
ImageNet	0.582 / 0.589	0.105 / 0.107	– / –
Retina	0.905 / 0.902	0.416 / 0.408	– / –
mass41	0.971 / 0.968	0.413 / 0.408	– / –
MetRef	0.826 / 0.835	0.411 / 0.420	– / –
MNIST	0.940 / 0.938	0.456 / 0.451	– / –
Tabula Muris	0.934 / 0.922	0.613 / 0.599	– / –
USPS	0.889 / 0.882	0.358 / 0.354	– / –
FlowRepository	– / 0.961	– / 0.442	– / –

Table S6: Support-width sensitivity in campaign 20260930T203319Z. Each median uses the same data sets with valid timing and quality at both support widths. Times cover precomputed-neighbor t-SNE calls with 250 exaggerated and 500 normal iterations, not matrix workflows. KL values describe different affinity objectives and are not matched-objective ratios.

Backend	P	k/P	Data sets	Seconds	Trust	Preserve@30	KL
CPU	10	1	4	7.76	0.914	0.424	2.194
CPU	10	3	4	8.27	0.897	0.410	2.046
CUDA	10	1	4	0.05	0.920	0.431	2.161
CUDA	10	3	4	0.07	0.907	0.416	2.084
CPU	20	1	4	7.62	0.935	0.449	2.063
CPU	20	3	4	8.44	0.921	0.429	1.846
CUDA	20	1	4	0.06	0.935	0.449	2.038
CUDA	20	3	4	0.09	0.927	0.434	1.951
CPU	30	1	10	54.67	0.944	0.443	1.992
CPU	30	3	10	54.94	0.933	0.430	1.735
CUDA	30	1	11	0.29	0.952	0.442	1.987
CUDA	30	3	11	0.69	0.940	0.433	1.727

Table S7: CPU full-workflow strong scaling in campaign 20260930T203319Z. Speedups at 2–12 threads are relative to the one-thread elapsed time in the same row.

Data set	Method	1-core s	2-core	4-core	8-core	12-core
COIL20	UMAP	14.5	1.90	2.19	2.02	2.13
MNIST	UMAP	246.6	1.44	2.64	3.65	3.40
flow18	UMAP	632.8	1.10	1.87	2.03	2.37
imagenet	UMAP	4135.6	1.28	2.33	2.02	1.49
COIL20	t-SNE	19.0	1.88	2.16	1.95	2.05
MNIST	t-SNE	300.4	1.52	2.46	3.21	2.94
flow18	t-SNE	553.9	1.23	1.85	2.30	2.51
imagenet	t-SNE	4052.9	1.33	2.31	2.09	1.47

Table S8: Held-out transformation in campaign 20260930T203319Z. Each value is the median of data-set medians over successful seeds. Query/reference Preserve@30 is compared with joint embedding of the same queries; the reference is held fixed during transformation.

Method	Backend	Data sets	Transform s	Preserve@30	Joint	Label KNN	Ref. shift
t-SNE	CPU	11	64.684	0.371	0.393	0.784	0
UMAP	CPU	11	65.976	0.343	0.346	0.776	0
t-SNE	CUDA	11	0.621	0.374	0.392	0.786	0
UMAP	CUDA	11	0.543	0.351	0.340	0.780	0

Table S9: Landmark reconstruction over 11 data sets in campaign 20260930T203319Z. Each row is the median of data-set medians for a 20-percent landmark fraction. RSS covers isolated fit workers; – means insufficient device-memory evidence.

Method	Backend	Data sets	Time ratio	Preserve@30 Δ	Procrustes	RSS ratio	GPU ratio
t-SNE	CPU	11	0.41	-0.006	0.86	0.98	–
Fuzzy UMAP	CPU	11	0.34	-0.011	0.52	1.01	–
t-SNE	CUDA	11	0.77	-0.006	0.95	1.09	–
Fuzzy UMAP	CUDA	11	1.29	-0.006	0.47	1.07	–

Table S10: Binary-adjacency versus fuzzy UMAP in campaign 20260930T203319Z. Ratios and quality changes are diagnostic medians over completed data sets; binary UMAP changes the graph objective and is not a drop-in speed substitute.

Workflow	Data sets	Binary/fuzzy time	Preserve@30 change
r_cpu	9	1.986	0.003
r_cuda	11	1.053	0.008

Table S11 separates complete-worker host RSS from sampled device-wide CUDA memory above the pre-worker allocator baseline. Device sampling may miss short peaks; missing samples remain missing. These memory boundaries are not the direct-Python fit-only timing boundary.

Table S11: Observed peak memory in campaign 20260930T203319Z (diagnostic, failed audit). Host RSS covers the complete worker, including data loading and quality assessment; it is not fit-only memory. CUDA values are sampled device-memory increases above the pre-worker baseline (nominal 0.2-s polling plus query time), so short peaks may be underestimated. Values are GiB; – indicates no valid measurement. Counts show valid data sets for each memory metric; missing measurements are archived separately.

A. Host peak RSS

Method	Completed	Measured	Median	Maximum
fastEmbedR PCA [CPU]	11	11	1.66	36.90
irlba [CPU]	11	11	1.80	33.98
fastEmbedR PCA [CUDA]	11	11	1.85	37.08
scikit-learn PCA [Python]	11	11	0.80	11.86
RAPIDS cuML PCA [CUDA]	11	11	1.10	18.95
Rtsne [CPU]	9	9	1.38	4.34
FIt-SNE [CPU]	11	11	1.75	34.53
fastEmbedR t-SNE [CPU]	11	11	1.53	22.47
fastEmbedR t-SNE [CUDA]	10	10	1.51	21.20
scikit-learn t-SNE [Python]	7	7	0.76	1.12
openTSNE [Python]	10	10	1.49	21.46
RAPIDS cuML t-SNE [CUDA]	10	10	1.17	18.78
umap [CPU]	7	7	5.32	9.00
uwot [CPU]	11	11	1.68	36.66
uwot fast SGD [CPU]	11	11	1.70	36.66
fastEmbedR fuzzy UMAP [CPU]	11	11	1.50	25.21
fastEmbedR binary UMAP [CPU]	11	11	1.49	25.21
fastEmbedR fuzzy UMAP [CUDA]	10	10	1.42	21.13
fastEmbedR binary UMAP [CUDA]	10	10	1.42	21.13
umap-learn [Python]	10	10	2.09	51.67
RAPIDS cuML UMAP [CUDA]	10	10	1.47	16.00

B. CUDA device-memory increase

Method	Completed	Measured	Median	Maximum
fastEmbedR PCA [CUDA]	11	0	–	–
RAPIDS cuML PCA [CUDA]	11	0	–	–
fastEmbedR t-SNE [CUDA]	10	7	1.03	17.79
RAPIDS cuML t-SNE [CUDA]	10	7	1.28	16.97
fastEmbedR fuzzy UMAP [CUDA]	10	7	0.78	13.57
fastEmbedR binary UMAP [CUDA]	10	7	0.76	13.57
RAPIDS cuML UMAP [CUDA]	10	7	0.73	11.96

CPU and CUDA PCA were compared with dense centered singular value decomposition on 11 fixed subsets. Signs and equivalent subspace rotations were aligned before score and loading comparisons. **irlba** is an approximate performance comparator, not the numerical reference. ImageNet CPU rank-2 score agreement was 0.783, an unresolved outlier for that configuration.

Graph construction was saved separately from clustering. Only COIL20 and MetRef completed the latest same-graph clustering validation; the other nine data-set jobs failed, so Table S13 is restricted to six seed runs per implementation. Native CPU Louvain, Leiden, and Walktrap were compared with **igraph**; CUDA Louvain and Leiden used the corresponding native CPU memberships as a reference. Label adjusted Rand index compares the recovered communities

Table S12: PCA agreement with dense centered SVD on fixed subsets in campaign 20260930T203319Z. The effective rank may be below the requested rank.

Backend	Requested rank	Data sets	Score correlation	Loading cosine
CPU	2	11	0.999972	0.999998
CPU	50	11	0.999292	0.998447
CUDA	2	11	1.000000	1.000000
CUDA	50	11	1.000000	1.000000

with the data-set labels; paired adjusted Rand index compares two partitions. These are not end-to-end embedding speedups, and Metal clustering is not represented.

Table S13: Clustering in campaign 20260930T203319Z. The Runs column counts completed data-set–seed fits; times exclude graph construction. Label ARI uses external data-set labels; paired ARI compares native CPU with igraph or an accelerator with native CPU. Graph sizes identify any fixed Walktrap subset. Missing runs are not zero.

Method	Backend	Implementation	Runs	n range	Median seconds	Label ARI	Paired ARI
louvain	CPU	fastEmbedR	6	873–1440	0.006	0.496	0.838
leiden	CPU	fastEmbedR	6	873–1440	0.011	0.518	0.965
louvain	CPU	igraph	6	873–1440	0.017	0.512	0.838
leiden	CPU	igraph	6	873–1440	0.025	0.515	0.965
louvain	CUDA	fastEmbedR	6	873–1440	0.026	0.504	0.911
leiden	CUDA	fastEmbedR	6	873–1440	0.045	0.513	0.933
louvain	METAL	fastEmbedR	0	–	–	–	–
leiden	METAL	fastEmbedR	0	–	–	–	–
walktrap	CPU	fastEmbedR	6	873–1440	0.320	0.424	0.922
walktrap	CPU	igraph	6	873–1440	0.091	0.428	0.922

Table S14: Reserved frozen-source resident same-graph clustering validation across the available labeled data sets. Metal rows use resident stored graphs, not unsupported file-backed full-graph Metal clustering. The denominator includes attempted failures. Adjusted Rand index (ARI) compares each membership with genuine source labels; paired ARI against igraph is separate. XXX denotes an unaudited result.

Method	Backend	Completed/attempted	Time (s)	Modularity	Label ARI	Paired ARI
Louvain	CPU	XXX/XXX	XXX	XXX	XXX	XXX
Leiden	CPU	XXX/XXX	XXX	XXX	XXX	XXX
Walktrap	CPU	XXX/XXX	XXX	XXX	XXX	XXX
Louvain	CUDA	XXX/XXX	XXX	XXX	XXX	XXX
Leiden	CUDA	XXX/XXX	XXX	XXX	XXX	XXX
Louvain	Metal	XXX/XXX	XXX	XXX	XXX	XXX
Leiden	Metal	XXX/XXX	XXX	XXX	XXX	XXX

The frozen PCA accuracy rerun must also resolve or retain the ImageNet CPU rank-2 outlier: aligned score agreement XXX under the same centered SVD reference, with elapsed time and peak memory from the tested build.

8. Experimental Deep1B file-backed study

The current design uses one source-locked row window of the Deep1B 96-variable float32 input. The first fitted row is source row 1,000,001; the 1- and 10-million-row cases use nested prefixes

of that window. This is a test of real-data execution and scaling, not an independent cross-data-set generalization experiment. Table S15 lists requested jobs, not completed observations. The ordinary 1-million-row fit and the file-backed 1-million-row fit are the only matched-input route comparison. An explicit full-row graph optimizer receives every row; it is not equivalent to landmark fitting. The candidate graph can still use approximate HNSW or IVF neighbors.

Table S15: Requested Deep1B evaluation arms. These are a protocol inventory, not speed or memory results. CPU counts denote allocated workers; a stage can use fewer effective workers.

Input route	Fitted rows	Allocation	Requested work
Ordinary matrix	1 million	CPU 4	PCA, t-SNE, UMAP
Ordinary matrix	1 million	CUDA 1	PCA, t-SNE, UMAP
File-backed full row	1, 10 million	CPU 4, 8, 16	PCA, KNN, t-SNE, UMAP, clustering
File-backed full row	1, 10 million	CUDA 1	PCA, KNN, t-SNE, UMAP
File-backed full row	1, 10 million	CUDA 2	PCA, KNN, t-SNE, UMAP, Leiden, Louvain

For every file-backed case, the output-row count and finite coordinates must be checked against the requested fitted rows. A separate 100-million- row, two-GPU case and a final billion-row case remain gated on an audited end-to-end restart and a quota-aware storage preflight; neither belongs to the current matrix. The 100-million-row input window is 38.4 GB, which alone cannot establish a source matrix beyond GPU or host memory. Output and intermediate file sizes must be recorded separately from the persistent source file. Full-row rasterized density plots are descriptive and cannot establish neighborhood quality. Deep1B has no class labels in this protocol, so neither invented labels nor inferred communities color the embedding. Quality is assessed on a fixed 4,096-row sample, and nearest-neighbor recall on 128 exact-reference query rows. The latter is a sampled check, not a guarantee for every graph edge. Current trustworthiness and Preserve@30 compare neighbors *within* the 4,096 sampled observations, not against all fitted rows. Full-reference neighborhood preservation remains a separate, unmeasured quantity; the two scopes must not be combined.

Execution audit, search quality, and hardware-use audit are distinct. The file-backed CPU study records effective stage-level core counts; the present CPU UMAP, Leiden, and Louvain optimizer calls request one worker even in four-, eight-, or 16-core allocations. Their allocation comparison is not an optimizer strong-scaling experiment. Device-wide memory samples must name each allocated GPU and distinguish its baseline from its peak. Two-GPU PCA must distinguish distributed fitting from score projection. The coordinated multi-GPU optimizers require per-device work and quality checks; allocation alone is not proof of acceleration. An incomplete multi-device memory trace is a measurement failure even when the PCA fit itself finishes. A plot generated by an otherwise failed job remains diagnostic.

The measured KNN stage includes its exact-reference audit. Build, query, and audit times are stored separately; its combined stage time must not be compared with a search-only comparator. The CPU HNSW record must include effective per-shard M, `efConstruction`, and `efSearch`. The fixed 128-query sample is not an untouched tuning holdout. A 0.99 recall claim requires its stated statistic and a separate holdout check; an execution `PASS` with a recall warning is not a quality pass.

The source-locked extended campaign did not pass as a whole: its file-backed audit recorded 18 passes, five failures, and 21 blocked cases. Within it, the Deep1B CPU boundary-PCA case and the paired flow18 full-versus-landmark cases passed their individual audits. They are reported below as scoped observations, not as a matched Deep1B workflow or a multi-GPU

result. Coordinated multi-GPU optimization is requested but not yet a validated performance result. No beyond-physical-host-RAM full-method test passed.

The Deep1B boundary case read the 384,000,000,008-byte input file and wrote 8,000,000,000 bytes of two-column float32 PCA scores, exactly one billion row slots. Its elapsed time was 6,924.096 seconds; measured peak process RSS was 451,850,240 bytes (431 MiB). Slurm recorded a 48 GiB memory request for the completed job. Seven dispersed output rows, including the first and last, were finite; this is not an exhaustive finite-value audit. The node reported 403.6 GB of physical RAM, larger than the source file, so the result shows execution within a smaller requested job allocation, not an input larger than physical host RAM. The PCA route alone completed; KNN, UMAP, t-SNE, and clustering did not complete on one billion rows.

The flow18 pair fitted the same 50,000 rows on one GPU, with 5,000 landmarks in the landmark arm. Preserve@30 compares neighborhoods within the same fixed 256 sampled rows, not against the full fitted reference. Both pairs passed the case-level source and quality-row identity audit.

Table S16: Source-locked, single-case flow18 full-versus-landmark diagnostic. GPU peaks are incremental observed MiB on device 0; no multi-seed inference or whole-campaign speed claim is implied.

Method	Route	Time (s)	RSS (MiB)	GPU (MiB)	Preserve@30
UMAP	Full	4.703	692	581	0.571
UMAP	Landmark	2.442	617	1128	0.552
t-SNE	Full	10.411	689	665	0.534
t-SNE	Landmark	2.982	734	976	0.541

These observations come from extended campaign `20261017T091542Z_jssrepair_CCu2_extended`, candidate image SHA-256 beginning `91387e35888b`, and package DLL SHA-256 beginning `5db59c0bfd57`. The landmark arms ran faster in this case but used more observed GPU memory; the 100,000-row, three-seed reviewer-closure experiment remains unmeasured.

8.1. Checkpoint and restart audit

The experimental full-graph kNN builders commit completed reference shards to a checkpoint sidecar. To test recovery independently of layout optimization, we generated the same 8,000-row, 16-variable float32 input for CPU HNSW and one-device CUDA IVF, with $k = 5$, four 2,000-row reference shards, 1,000-row query blocks, and seed 19. Each arm built a fresh checkpointed graph, killed a second process after a committed shard, and restarted it from the same partial files. The audit required the restart log to begin after the last committed shard, identical effective search settings, and byte-identical final neighbor-ID and float32-distance files. Exact-reference recall was sampled on 32 query rows. The 512-MB control constrained algorithm buffers, not process RSS.

Both recovered graphs matched their uninterrupted counterparts byte for byte. The candidate installed R library had package version 0.1 and DLL SHA-256 prefix `f341dcc75118`; the same synthetic input SHA-256 prefix was `c24ddaf96dd5` in both arms. The two pilot records, code hashes, effective settings, and full digests are retained with the benchmark scripts. Sampled fresh/restarted RSS was 511/499 MB on CPU and 861/845 MB on CUDA; CUDA VRAM was 409/273 MB. The RSS monitor sums process-tree resident values and can double-count shared

Table S17: Development kill-and-resume pilot on Chiamaka, not a Deep1B performance result. Seconds are external wall time including process startup. RSS and VRAM are sampled observed peaks, not strict maxima. The interrupted and resumed times are added; lower is better only for matched long-running work.

Backend	Reused shards	Fresh (s)	Killed (s)	Restart (s)	Total (s)	Recall@5
CPU HNSW	1/4	3.42	2.52	3.22	5.73	1.00
CUDA IVF	2/4	3.19	2.49	3.10	5.58	1.00

pages; these figures are diagnostic rather than a memory bound. The small test took longer to kill and resume than to rerun from scratch because restart and container startup outweighed saved shard work. It establishes restart correctness for these routes, not a recovery speedup or a 100-million-row claim. A final Deep1B experiment should kill at a stated completed shard, report checkpoint and partial-output bytes, compare total lost and reused work against a same-controls fresh run, and repeat on the frozen image. Recovery of a completed graph, layout optimizer, or multi-device job is outside this audited scope. The billion-row two-GPU case must not be submitted on the strength of this kNN-only restart test.

The following XXX tables and figure are editorial placeholders. A case that fails execution, recall, row coverage, or device monitoring receives its failure code rather than an imputed number. All row windows share the same first source row. The final comparison also requires the package, container, and benchmark-suite hashes for every case.

Table S18: Reserved Deep1B file-backed stage times in seconds. KNN is build plus query; exact-reference audit time is separate. Clustering shows Leiden/Louvain. XXX is pending and – is unsupported.

Rows	Allocation	PCA	KNN	t-SNE	UMAP	Clustering
1M	CPU 4	XXX	XXX	XXX	XXX	XXX/XXX
1M	CPU 8	XXX	XXX	XXX	XXX	XXX/XXX
1M	CPU 16	XXX	XXX	XXX	XXX	XXX/XXX
1M	CUDA 1	XXX	XXX	XXX	XXX	–
1M	CUDA 2	XXX	XXX	XXX	XXX	XXX/XXX
10M	CPU 4	XXX	XXX	XXX	XXX	XXX/XXX
10M	CPU 8	XXX	XXX	XXX	XXX	XXX/XXX
10M	CPU 16	XXX	XXX	XXX	XXX	XXX/XXX
10M	CUDA 1	XXX	XXX	XXX	XXX	–
10M	CUDA 2	XXX	XXX	XXX	XXX	XXX/XXX

Table S19: Reserved Deep1B validity and resource observations for the same rows as Table S18. GPU peak is the largest observed per-device value; final data must include each GPU. Scratch excludes the shared input and container. Recall is an exact-reference sample, not every graph edge. XXX is unverified.

Rows	Allocation	Fit (s)	RSS (GiB)	GPU (GiB)	Scratch (GiB)	Recall@30	Row audit
1M	CPU 4	XXX	XXX	–	XXX	XXX	XXX
1M	CPU 8	XXX	XXX	–	XXX	XXX	XXX
1M	CPU 16	XXX	XXX	–	XXX	XXX	XXX
1M	CUDA 1	XXX	XXX	XXX	XXX	XXX	XXX
1M	CUDA 2	XXX	XXX	XXX	XXX	XXX	XXX
10M	CPU 4	XXX	XXX	–	XXX	XXX	XXX
10M	CPU 8	XXX	XXX	–	XXX	XXX	XXX
10M	CPU 16	XXX	XXX	–	XXX	XXX	XXX
10M	CUDA 1	XXX	XXX	XXX	XXX	XXX	XXX

Rows	Allocation	Fit (s)	RSS (GiB)	GPU (GiB)	Scratch (GiB)	Recall@30	Row audit
10M	CUDA 2	XXX	XXX	XXX	XXX	XXX	XXX

Table S20: Reserved Deep1B embedding-quality check on fixed row IDs. T and P@30 use a fixed within-sample reference; Full-P@30 compares those queries with all fitted rows and therefore needs a separate validated neighbor search. XXX is unverified, not a quality pass.

Rows	Backend	Method	T	P@30	Full-P@30	Stress
1M	CPU 4	PCA	XXX	XXX	XXX	XXX
1M	CPU 4	t-SNE	XXX	XXX	XXX	XXX
1M	CPU 4	UMAP	XXX	XXX	XXX	XXX
1M	CUDA 1	PCA	XXX	XXX	XXX	XXX
1M	CUDA 1	t-SNE	XXX	XXX	XXX	XXX
1M	CUDA 1	UMAP	XXX	XXX	XXX	XXX
10M	CPU 4	PCA	XXX	XXX	XXX	XXX
10M	CPU 4	t-SNE	XXX	XXX	XXX	XXX
10M	CPU 4	UMAP	XXX	XXX	XXX	XXX
10M	CUDA 1	PCA	XXX	XXX	XXX	XXX
10M	CUDA 1	t-SNE	XXX	XXX	XXX	XXX
10M	CUDA 1	UMAP	XXX	XXX	XXX	XXX

Table S21: Reserved multi-GPU work and quality audit, paired with the one-GPU row of the same size and source window. Time ratio is one-GPU divided by multi-GPU complete-fit time. Quality differences are multi-GPU minus one-GPU within-sample trustworthiness/Preserve@30. Per-device work and memory must name every allocated GPU.

Rows	GPUs	Active/allocated	Work by GPU	Peak GiB by GPU	Time ratio	Quality Δ (T/P@30)
1M	2	XXX	XXX	XXX	XXX	XXX/XXX
10M	2	XXX	XXX	XXX	XXX	XXX/XXX

The corrected holdout auditor passed its prespecified *mean*-recall gate. It did not establish a per-query 0.99 guarantee: the selected holdout had 200 of 8,192 query rows below 0.99, although its tenth percentile recall was 1. The two insertion orders are diagnostic, not independent data sets. The results do not validate a billion-row graph or the end-to-end embedding quality. The original audit had a numeric format/ranking defect; only its corrected v2 result is used here.

The full-versus-landmark comparison must not substitute the earlier 50,000-row diagnostic for the independently scheduled 100,000-row flow18 cases. The new arm draws one fixed random source sample (sampling seed 20261018), uses fit seeds 4, 17, and 42 and a 20-percent landmark fraction, and holds the 1,024 evaluation IDs fixed within each pair. It records all-row finiteness and checksums, complete-call time, peak RSS, sampled per-device VRAM, trustworthiness, and Preserve@30. An identical layout across seed requests is reported as no observed seed variation, not three independent stability observations. The restart analysis must compare completed coordinates or quality under a declared numerical tolerance and record additional wall time and disk writes. The current fixed 128-row KNN audit cannot fill the untouched holdout rows in Table S22.

A separate candidate-image preflight completed on 100,000 synthetic rows and two L40S devices. It checked full-row finite PCA, t-SNE, and UMAP outputs, observed work on both devices, and sampled exact-neighbor recall of 1. This checks route execution, not Deep1B

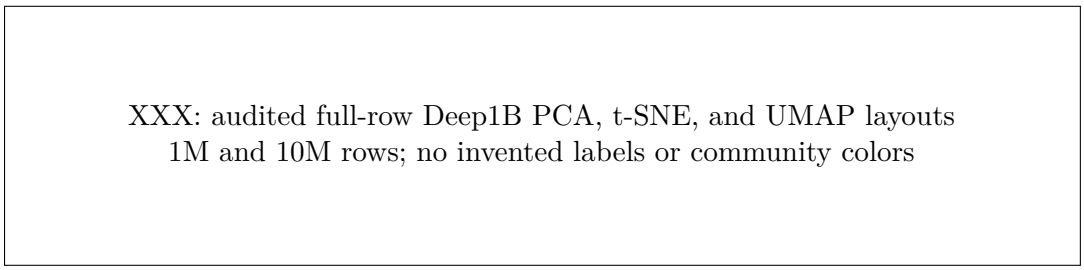

Figure S4: Reserved full-row visualization panel. The eventual rasterized views must represent every fitted row; row count and finite-coordinate audits, not pixels, establish output coverage. The box contains no measured embedding.

Table S22: Deep1B five-million-row HNSW pilot and disjoint-query holdout. Case 2 was selected using tuning queries only, among four measured candidate/order runs, by projected build-plus-query time subject to mean recall@30 of 0.99. Each split contains 8,192 exact-reference queries. The last row uses the same settings with a new insertion order. Build/query are pilot-run seconds, not a billion-row timing. Turing remains unavailable because its source input is absent.

Data	Evaluation	M	ef_C	ef_S	Mean recall	Minimum	Rows < 0.99	Build/query (s)
Deep1B	Tune	24	160	45	0.99768	0.733	199	250.1/35.8
Deep1B	Holdout	24	160	45	0.99750	0.667	200	250.1/35.8
Deep1B	Holdout, new order	24	160	45	0.99747	0.700	198	264.0/38.9
Turing	Unavailable	–	–	–	–	–	–	–

Table S23: Reserved route-validation analyses. Full and landmark fits must use identical source rows and comparable seeds; the quality reference is stated in the final result. A bounded-RAM result requires an enforced allocation below the input payload and complete output coverage. XXX denotes an unrun or unaudited experiment, not success.

Data and test	Method	Time (s)	Peak RSS (GiB)	Preserve@30	Other audit
flow18 100k full, seeds 4/17/42	t-SNE	XXX	XXX	XXX	XXX
flow18 100k landmark, same seeds	t-SNE	XXX	XXX	XXX	XXX
flow18 100k full, seeds 4/17/42	UMAP	XXX	XXX	XXX	XXX
flow18 100k landmark, same seeds	UMAP	XXX	XXX	XXX	XXX
Deep1B 10M bounded RAM	Full UMAP	XXX	XXX	XXX	XXX
Deep1B 1M restart	Full UMAP	XXX	XXX	XXX	XXX
Deep1B 1M restart	Full t-SNE	XXX	XXX	XXX	XXX

quality, speedup, memory scaling, or four-device correctness. It is not pooled with the publication benchmark.

9. Campaign identity and unresolved gates

The source archive and the installed native library are recorded separately because this image included a container-build patch. Table S24 fixes the exact diagnostic source and benchmark identities. The campaign's final audit reported 34 failed or timed-out tasks and four output-audit failures. The replication directory retains every unsuccessful method–data-set status.

Table S24: Immutable identity of diagnostic campaign 20260930T203319Z. The campaign failed its final audit.

Item	Identifier
Package	fastEmbedR 0.1
Source commit	`7f22c52b28400535c2a246de68e7189cd9880ea1`
Source archive SHA-256	`a3bc4a09e6f60bb719cf49b5d91af876c084666ff1b390d06ea9c155d2e991b9`
Build patch SHA-256	`8a725d2d2579425ff8d7edd3684f74c3fbb9ec1e5e3624c53e3eb3280b1188c7`
Container SHA-256	`54e9604d951ecb3d80f49671d9db7a35cf19f1342eb3ee7c708cbc0c3a7f0aa8`
Installed library SHA-256	`d44f08b574a4932bd36f7afb1d0da0895ccaaa9e1e5ca3f4c040ef2357d649fc`
Benchmark manifest SHA-256	`d0dddde1d82fa3faa01a9617db55b4128f8a04de61b8fd4dfb6f117d99e6f44c`

The experimental file-backed phase was added after this image and campaign were created. The revised Deep1B matrix above covers route matching at one million rows and requests scaling to ten million rows. A resumable 100-million-row two-GPU fit and a final billion-row two-GPU fit remain separate, unsubmitted gates. The matrix does not supply a memory-boundary test, a matched full-versus-landmark analysis, or completed multi-GPU performance comparison. None of its observations is pooled with the diagnostic tables above; a new release-level comparison requires passing source, output, recall, and memory audits.

10. Embedding gallery

The gallery uses coordinates from campaign 20260929T210526Z, separate from the full-data timing campaign in Table S1. Only seed-4 layouts are shown. Other seeds contribute to numerical summaries but are not illustrated. That campaign also failed its audit; these plots are visual diagnostics, not release-level validation. Missing panels are labelled and are never filled from another campaign.

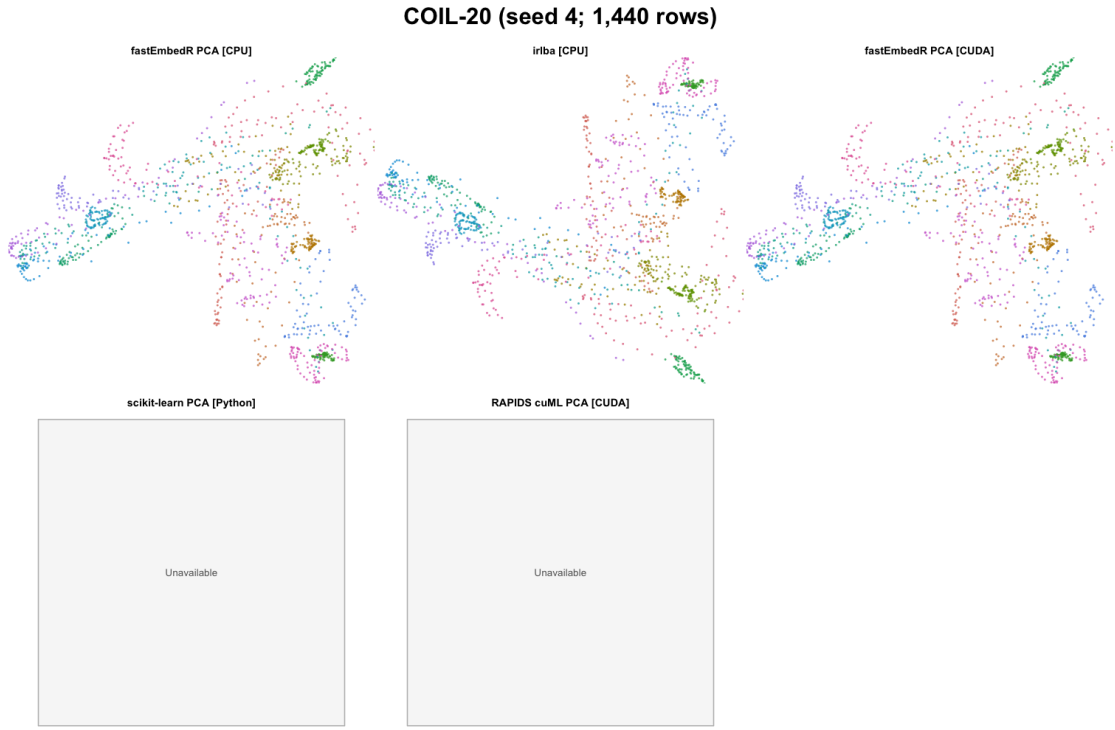

Figure S5: COIL-20 PCA outputs for the tested methods (seed 4; 1,440 rows). Colors encode benchmark labels and were not used for fitting.

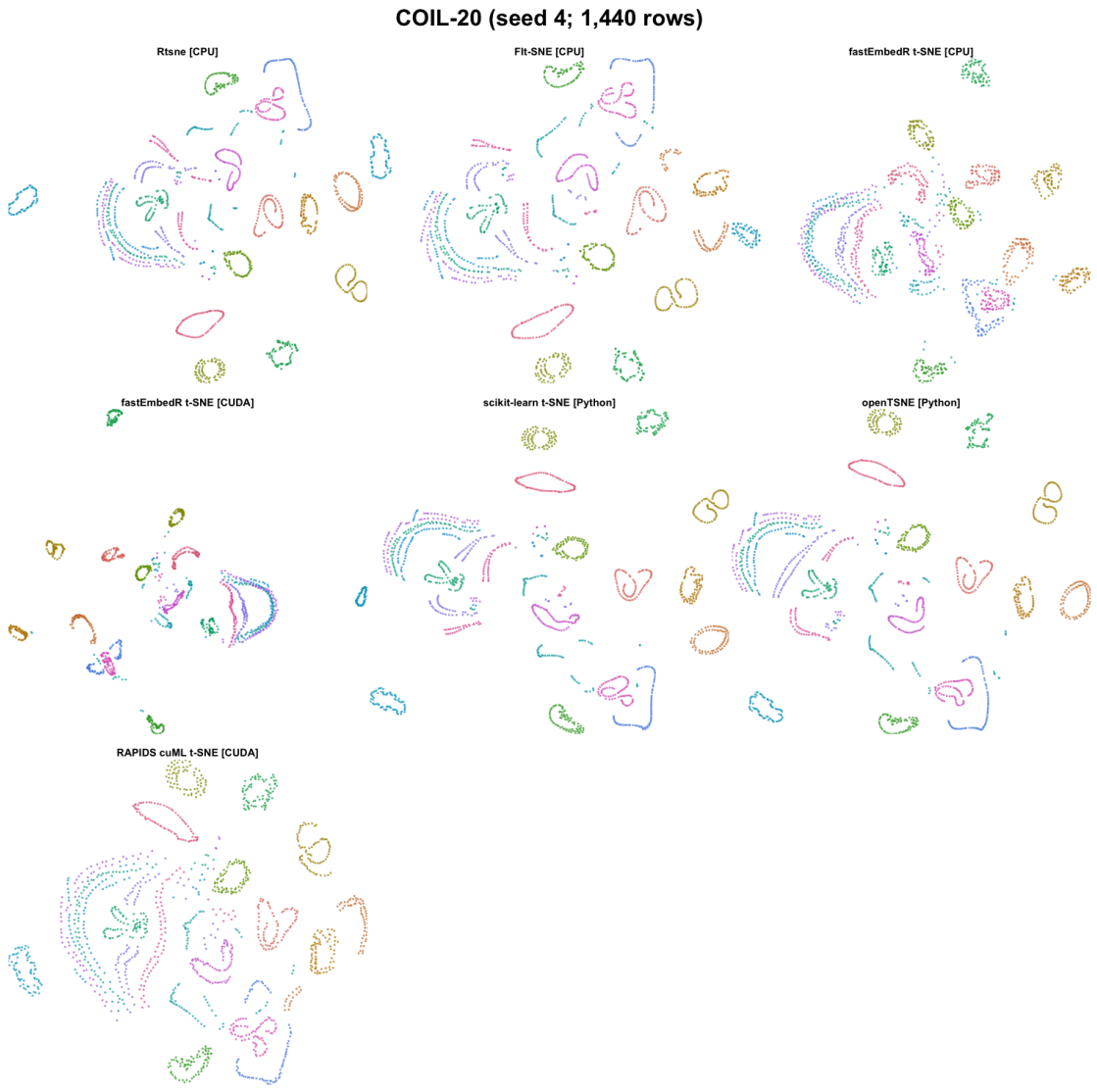

Figure S6: COIL-20 t-SNE outputs for the tested methods (seed 4; 1,440 rows). Colors encode benchmark labels and were not used for fitting.

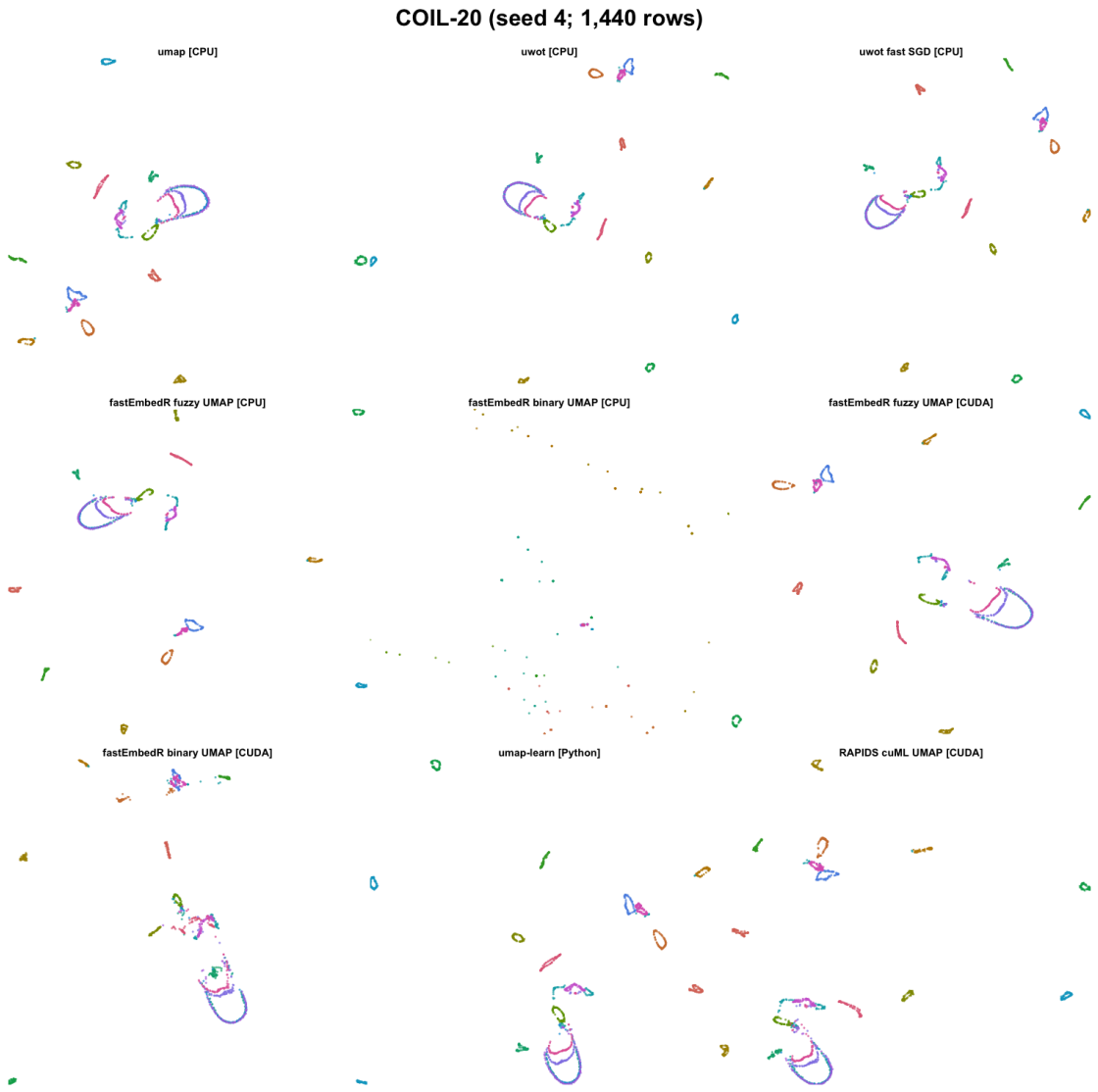

Figure S7: COIL-20 UMAP outputs for the tested methods (seed 4; 1,440 rows). Colors encode benchmark labels and were not used for fitting.

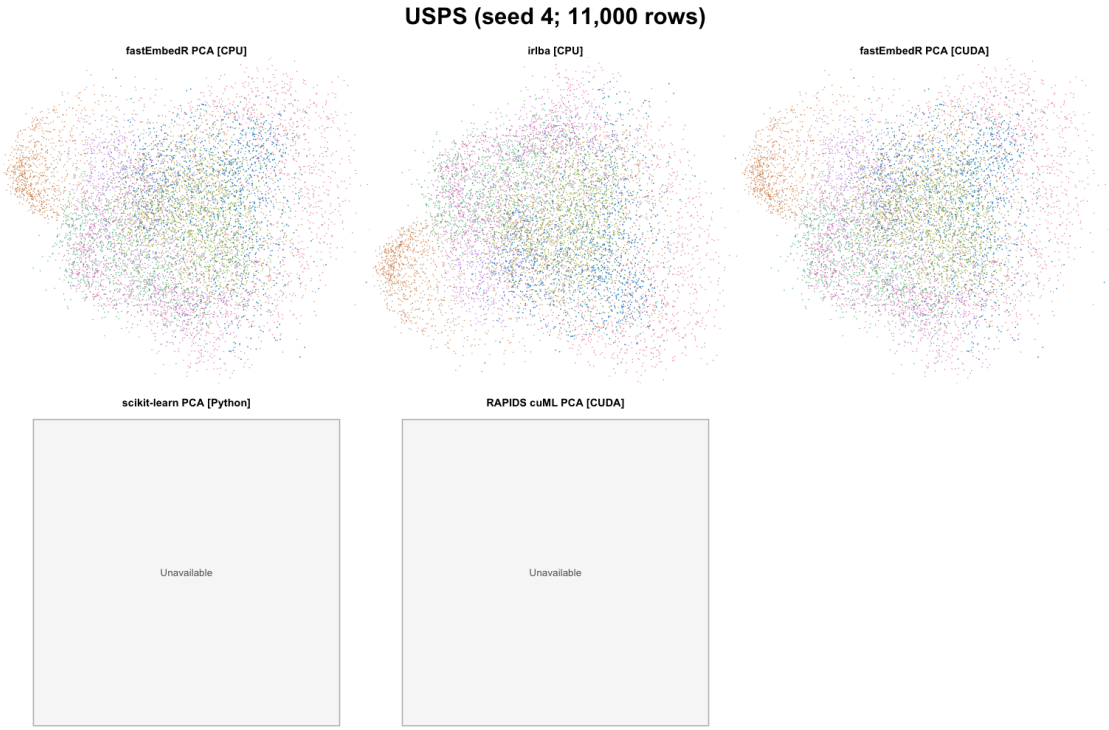

Figure S8: USPS PCA outputs for the tested methods (seed 4; 11,000 rows). Colors encode benchmark labels and were not used for fitting.

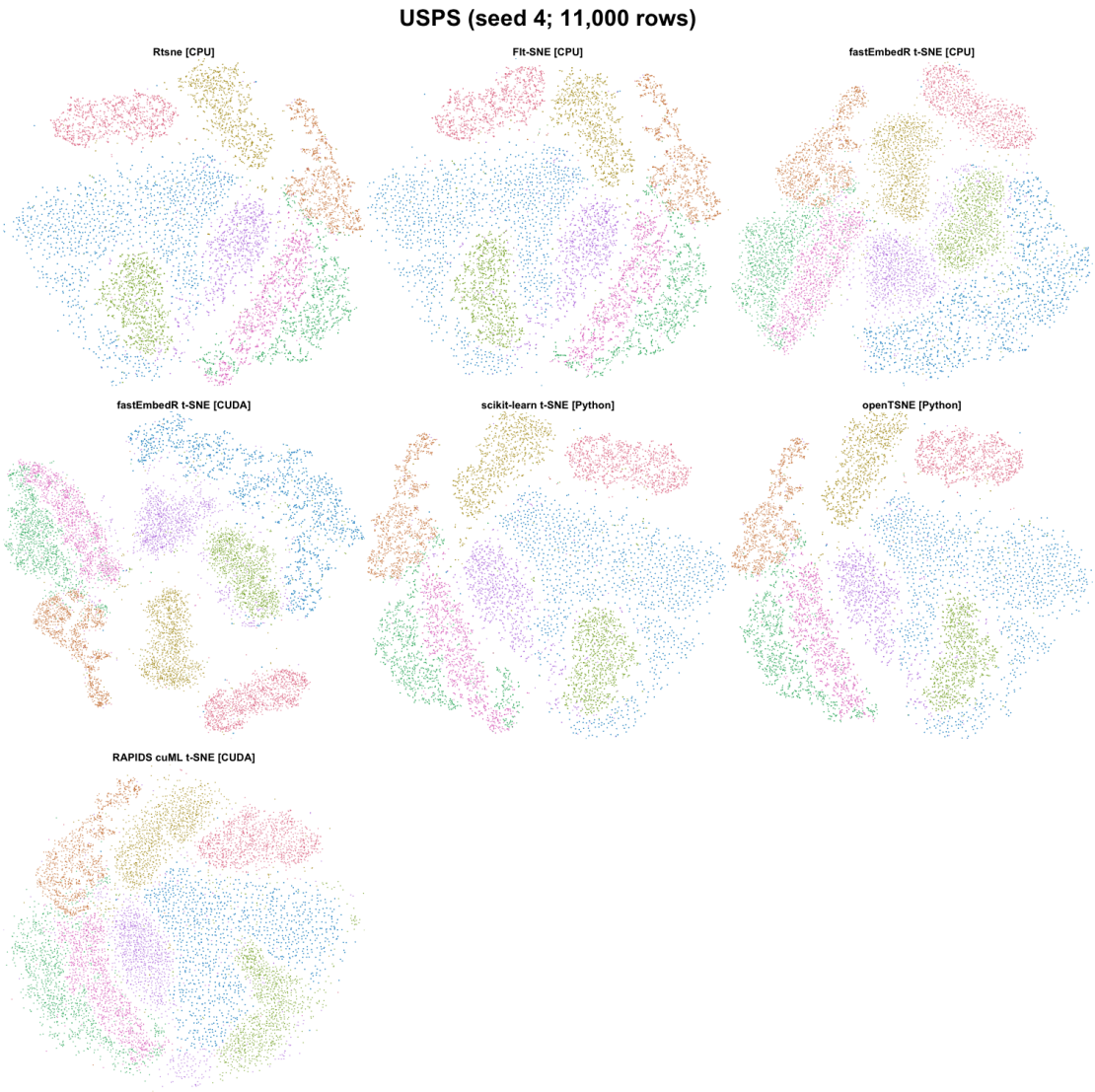

Figure S9: USPS t-SNE outputs for the tested methods (seed 4; 11,000 rows). Colors encode benchmark labels and were not used for fitting.

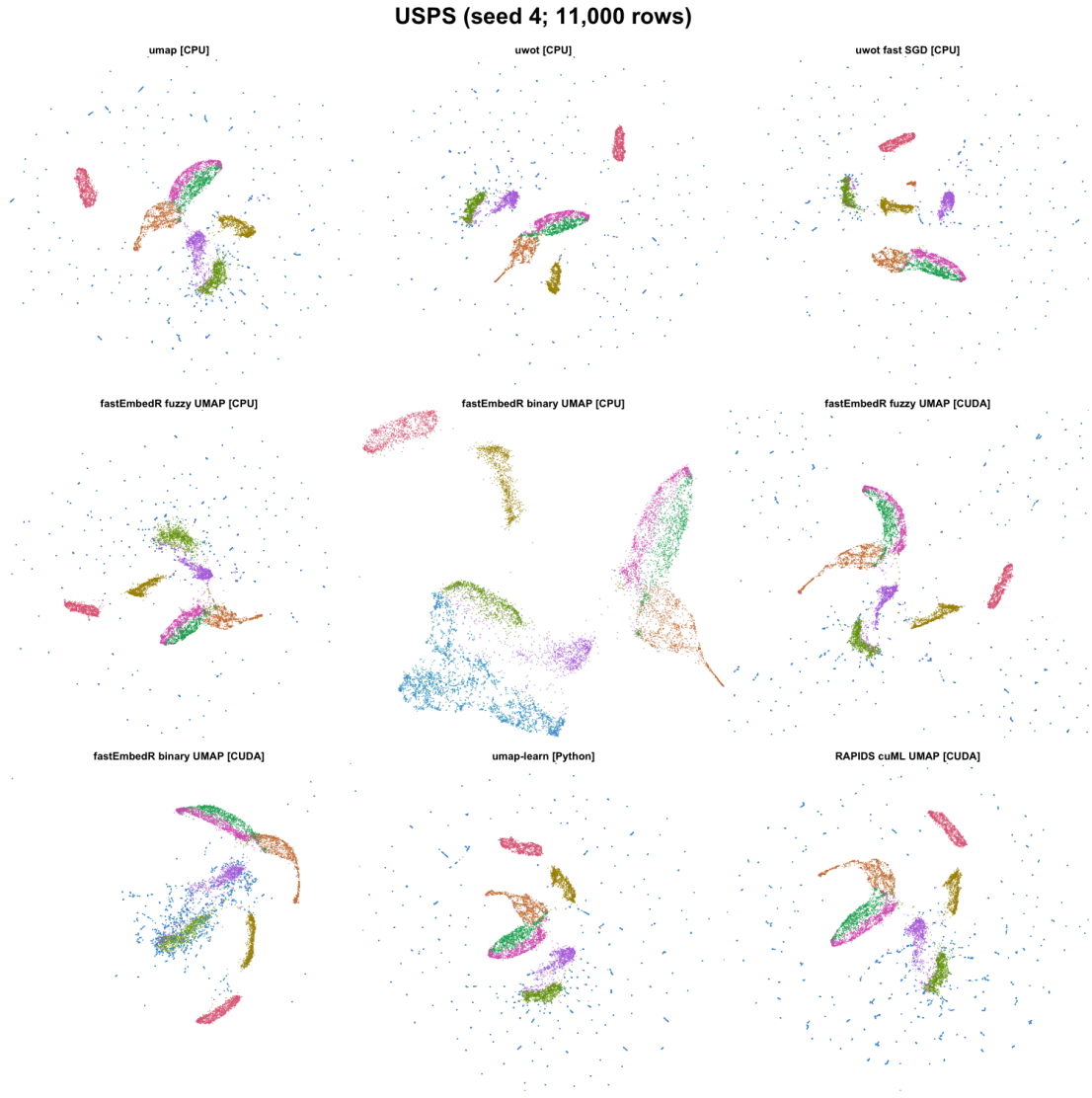

Figure S10: USPS UMAP outputs for the tested methods (seed 4; 11,000 rows). Colors encode benchmark labels and were not used for fitting.

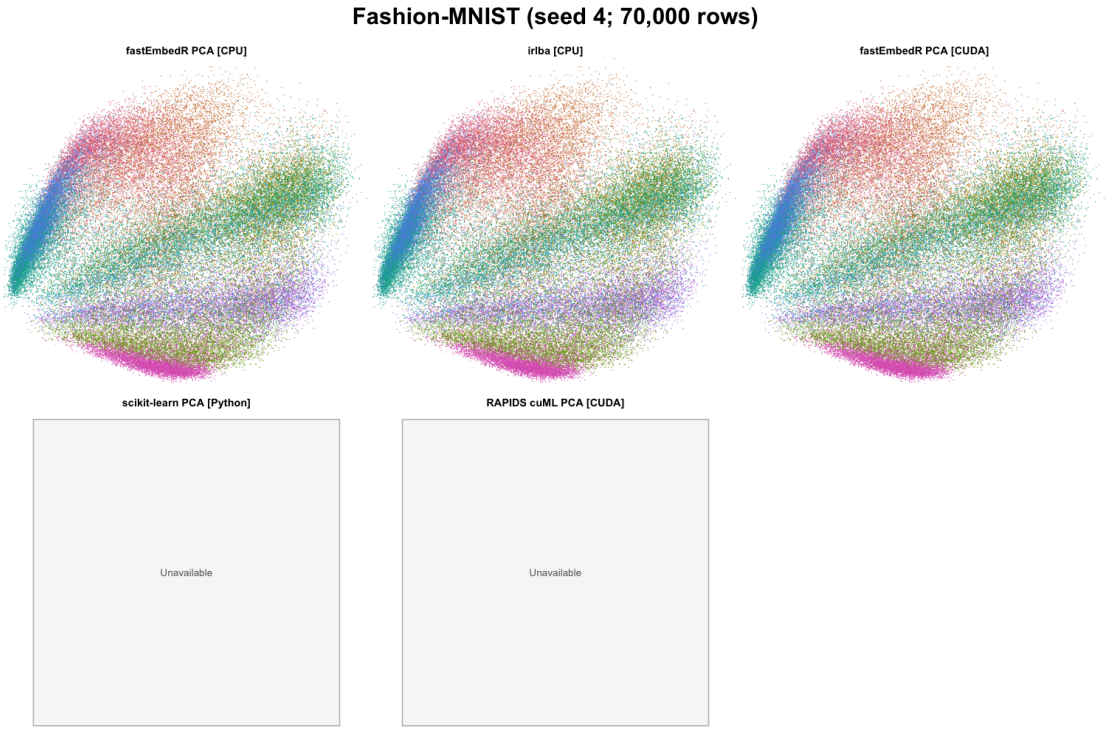

Figure S11: Fashion-MNIST PCA outputs for the tested methods (seed 4; 70,000 rows). Colors encode benchmark labels and were not used for fitting.

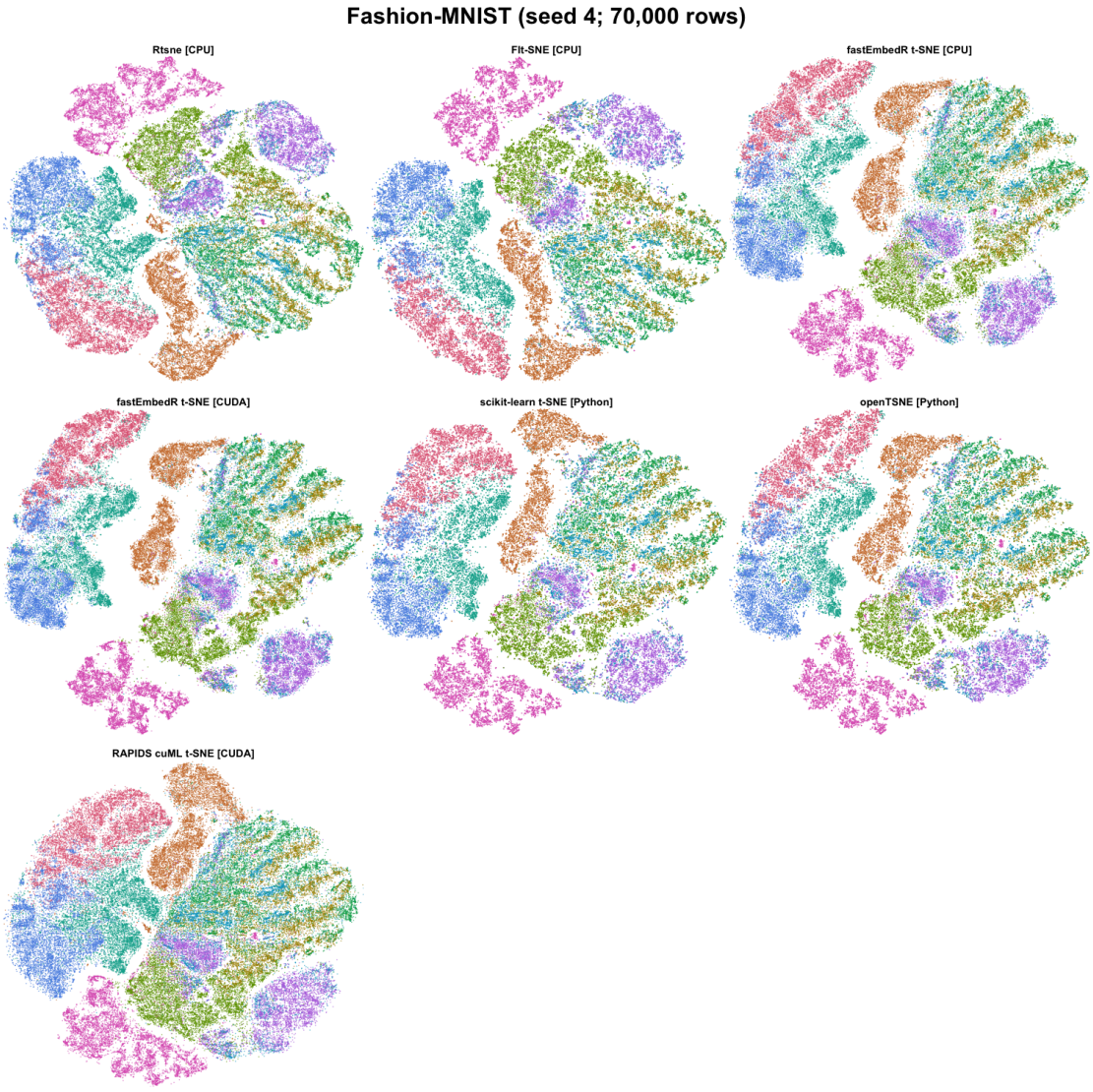

Figure S12: Fashion-MNIST t-SNE outputs for the tested methods (seed 4; 70,000 rows). Colors encode benchmark labels and were not used for fitting.

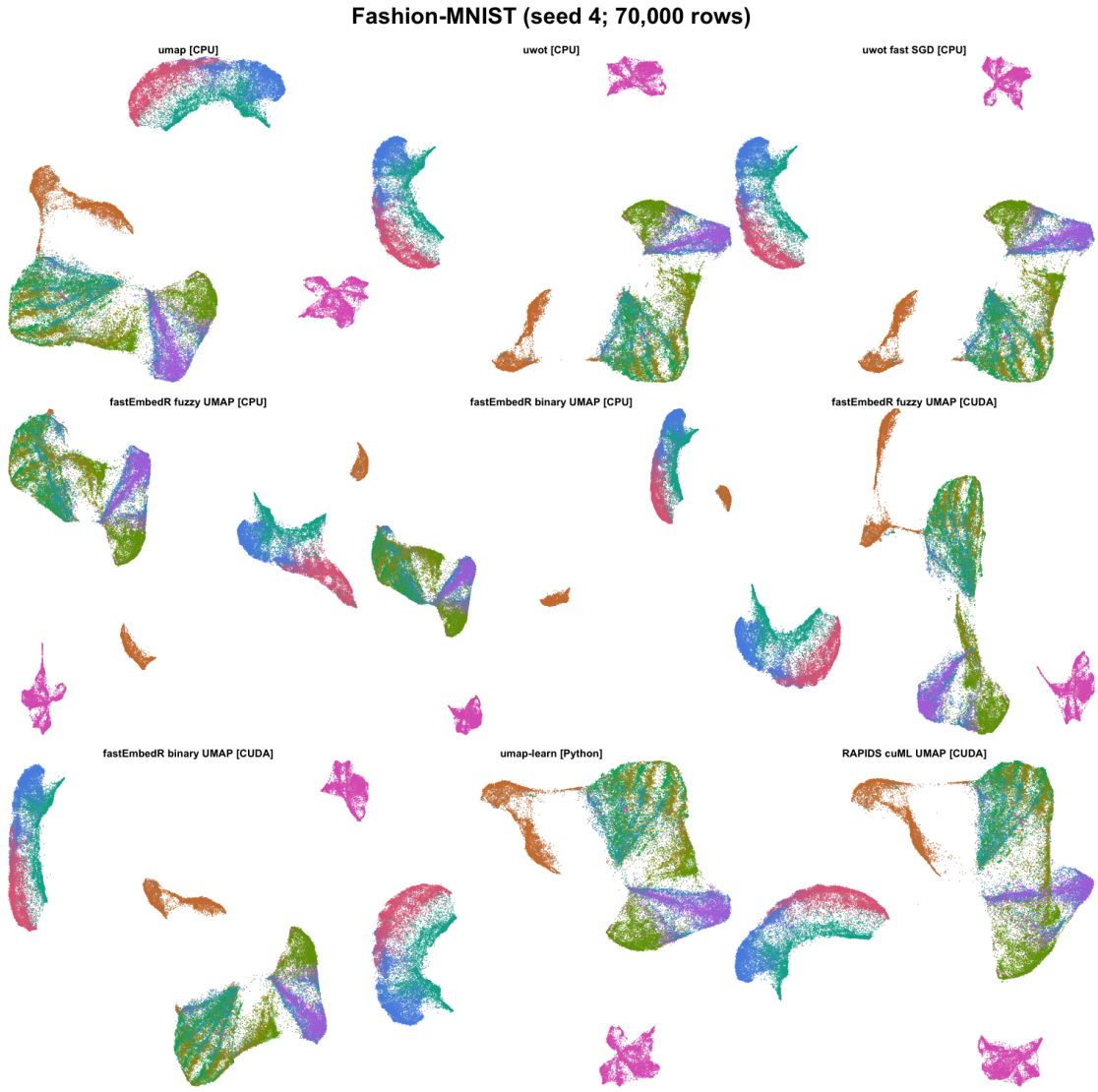

Figure S13: Fashion-MNIST UMAP outputs for the tested methods (seed 4; 70,000 rows). Colors encode benchmark labels and were not used for fitting.

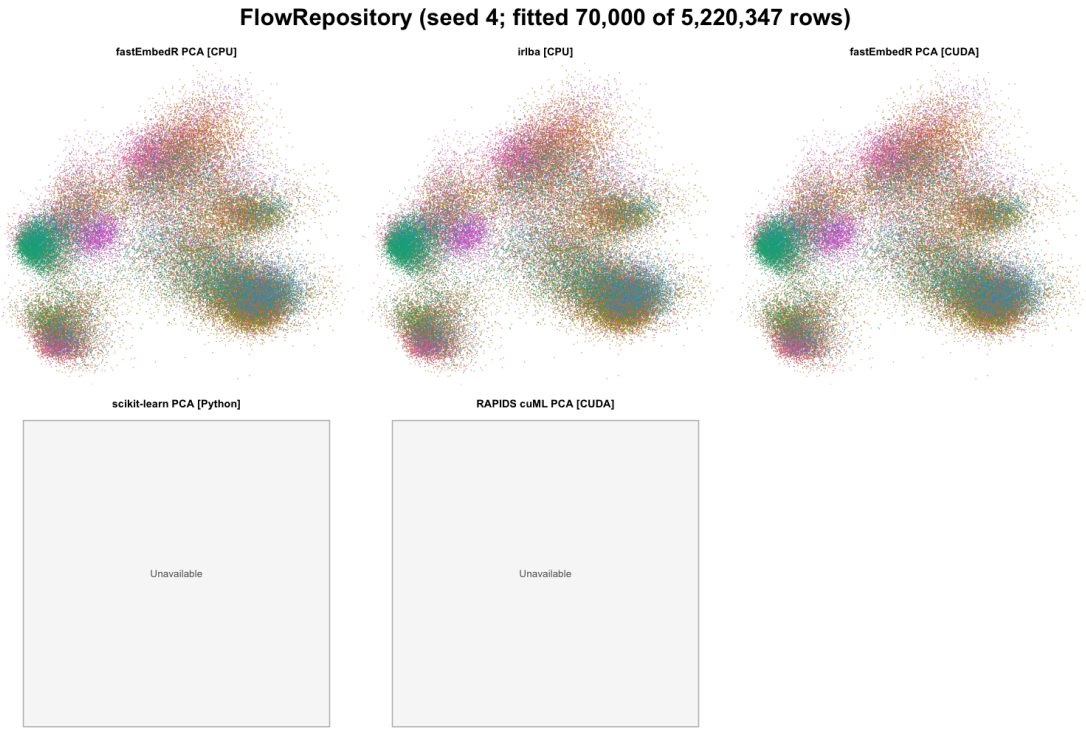

Figure S14: FlowRepository PCA outputs for the tested methods (seed 4; fitted 70,000 of 5,220,347 rows). Colors encode benchmark labels and were not used for fitting.

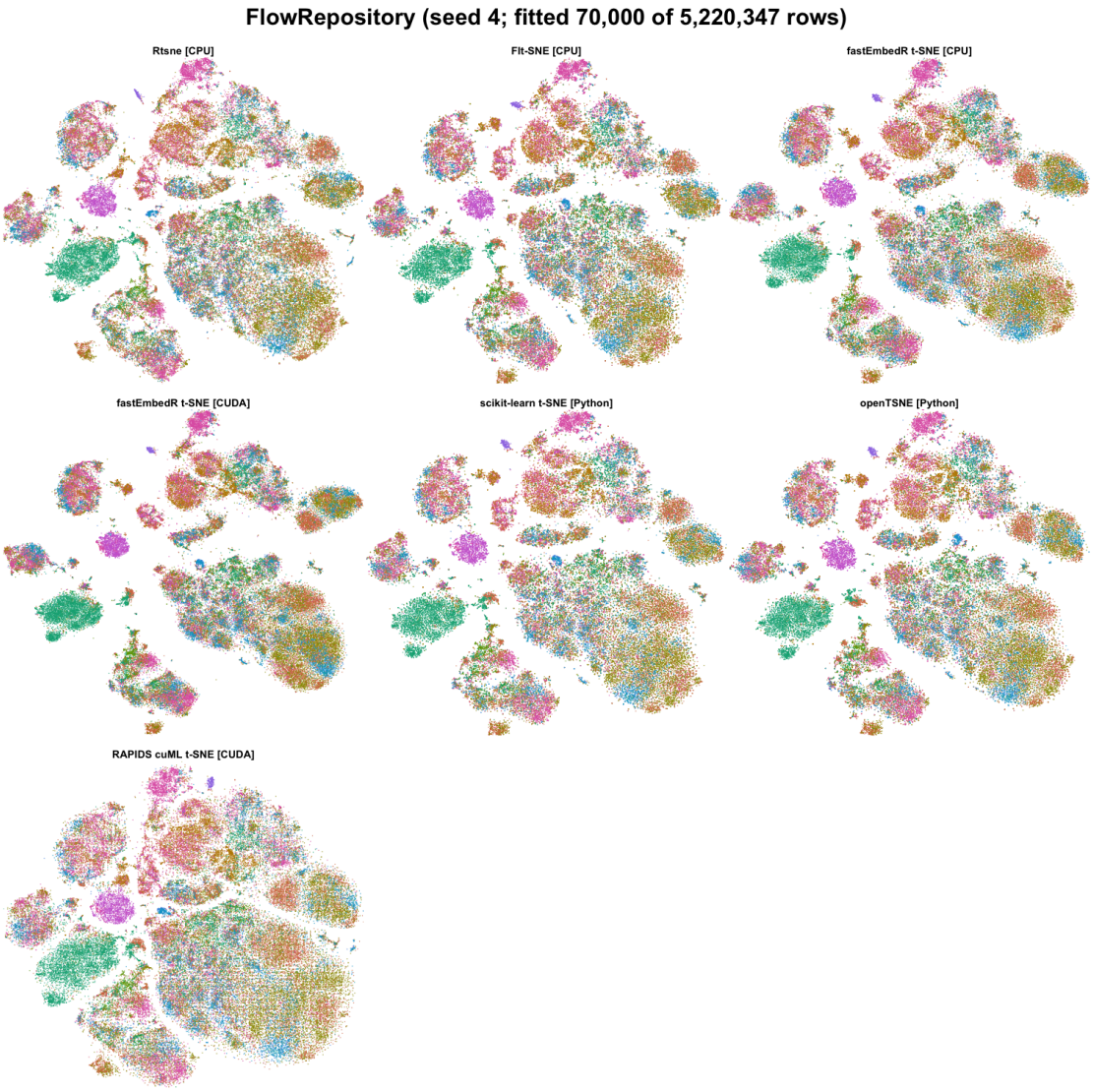

Figure S15: FlowRepository t-SNE outputs for the tested methods (seed 4; fitted 70,000 of 5,220,347 rows). Colors encode benchmark labels and were not used for fitting.

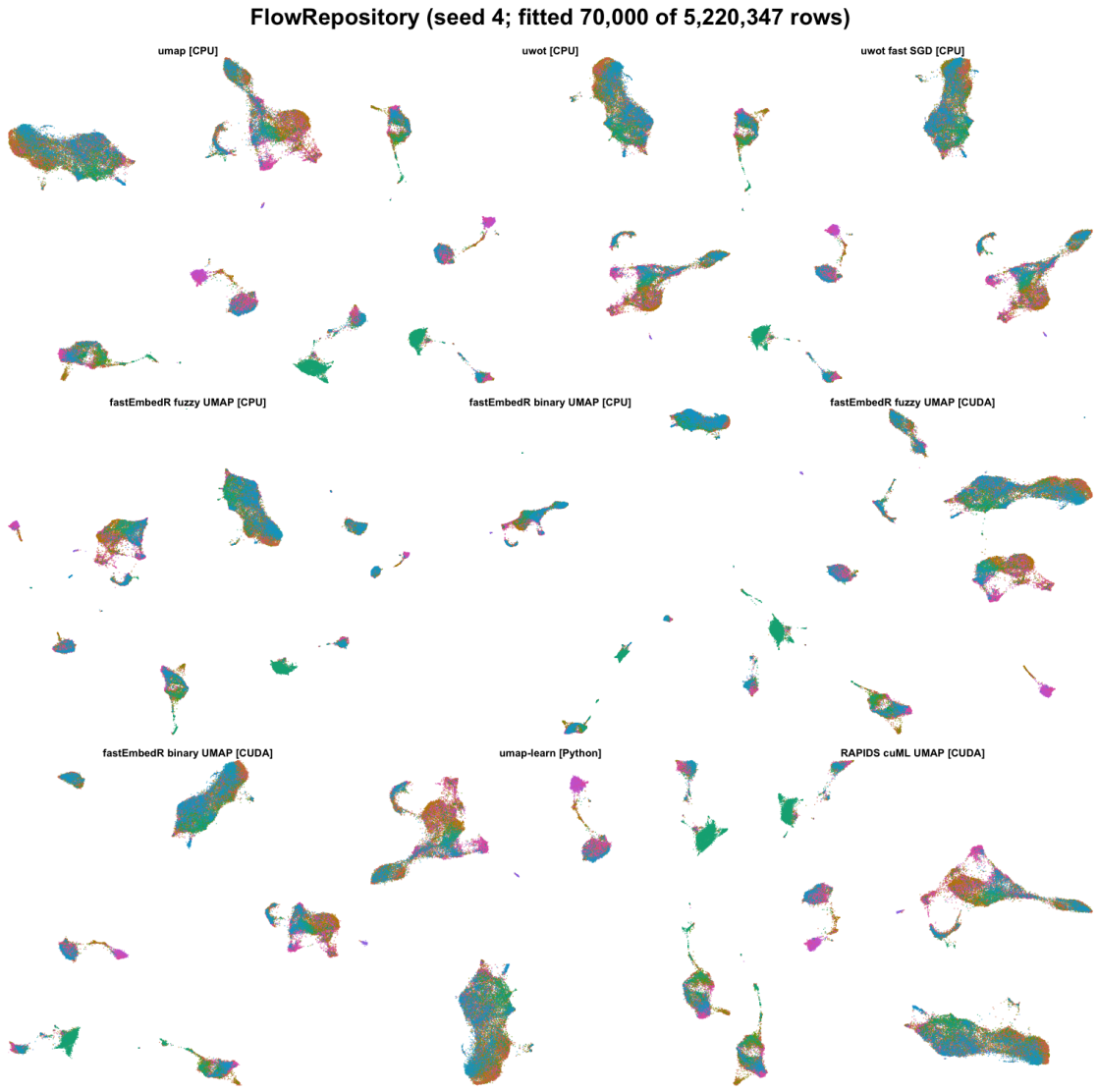

Figure S16: FlowRepository UMAP outputs for the tested methods (seed 4; fitted 70,000 of 5,220,347 rows). Colors encode benchmark labels and were not used for fitting.

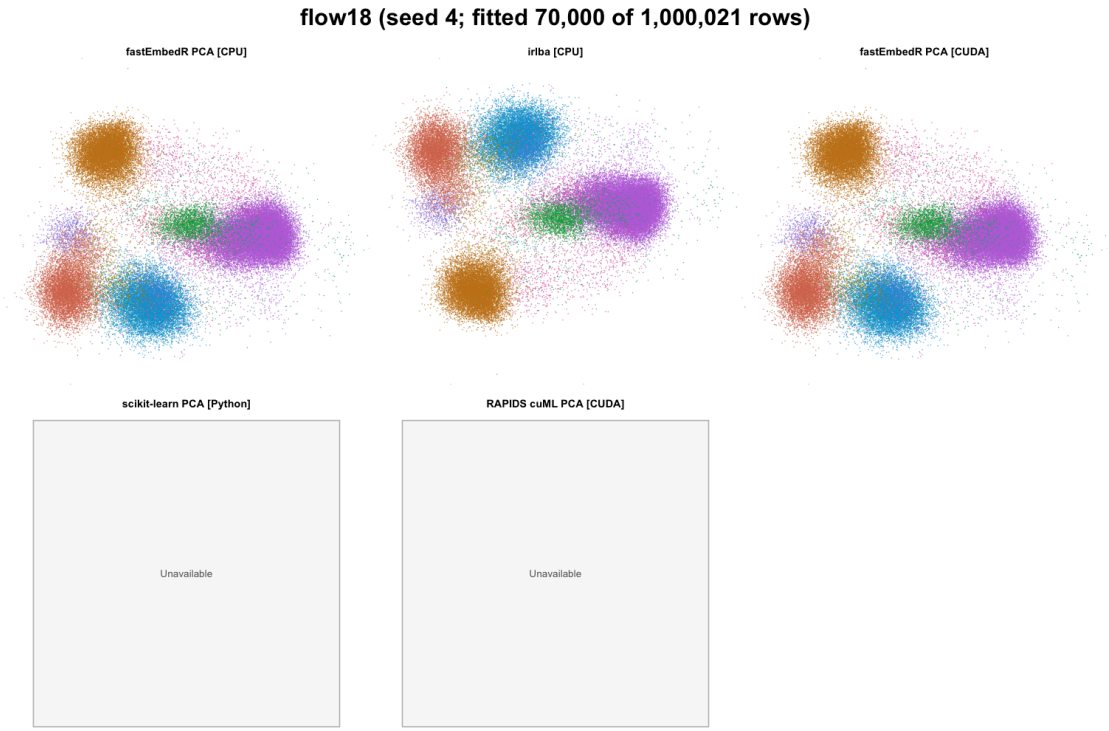

Figure S17: flow18 PCA outputs for the tested methods (seed 4; fitted 70,000 of 1,000,021 rows). Colors encode benchmark labels and were not used for fitting.

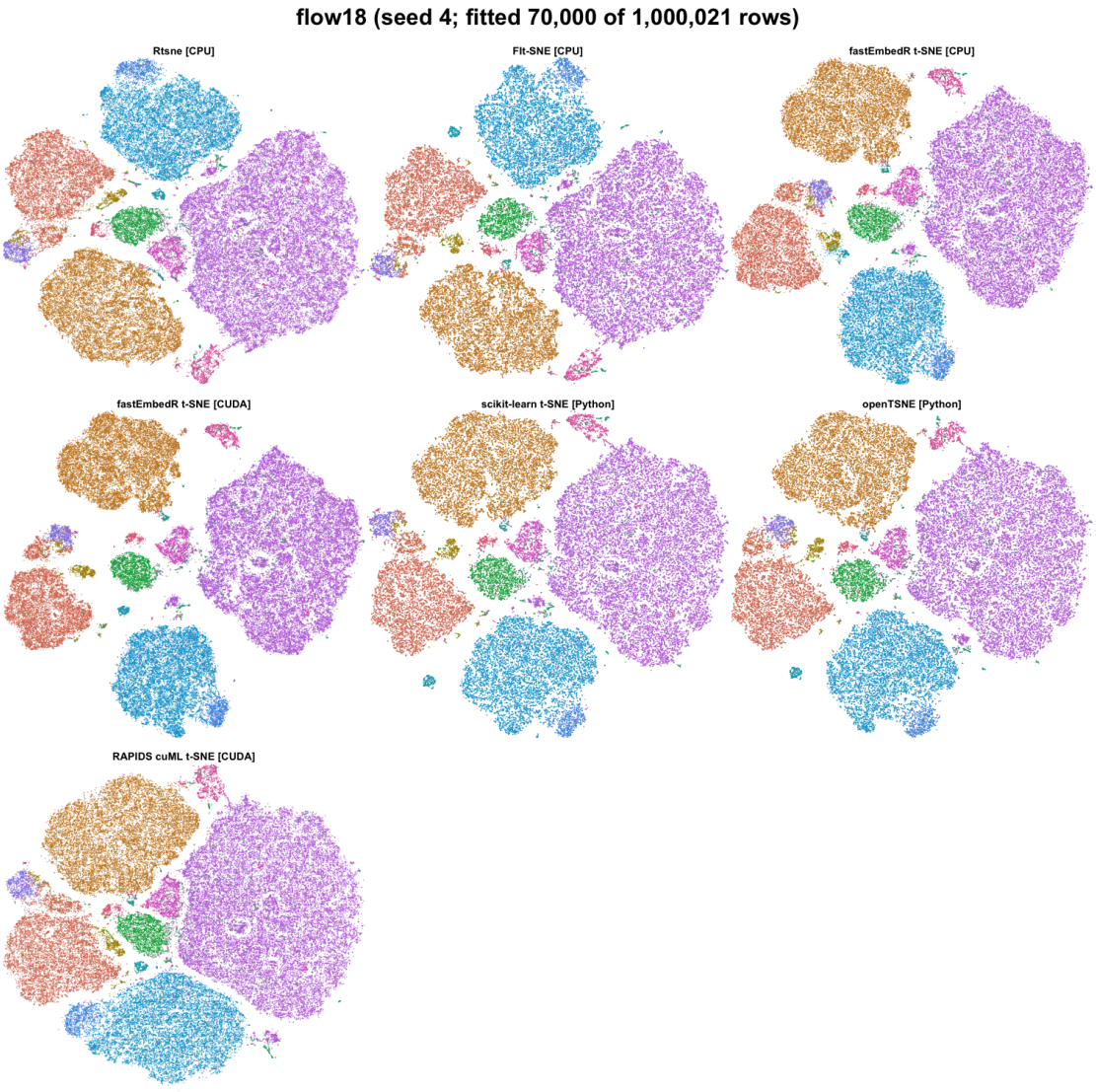

Figure S18: flow18 t-SNE outputs for the tested methods (seed 4; fitted 70,000 of 1,000,021 rows). Colors encode benchmark labels and were not used for fitting.

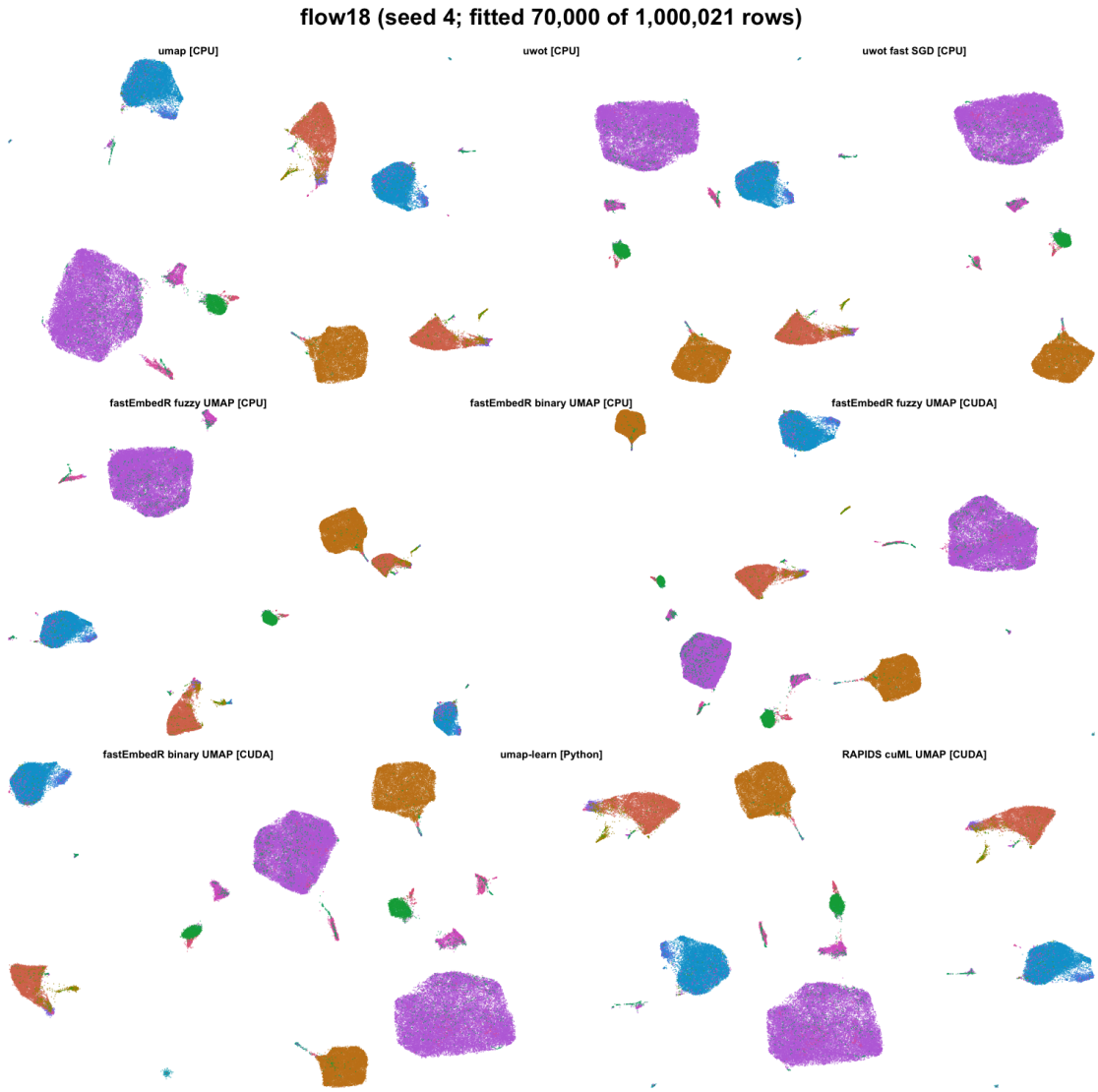

Figure S19: flow18 UMAP outputs for the tested methods (seed 4; fitted 70,000 of 1,000,021 rows). Colors encode benchmark labels and were not used for fitting.

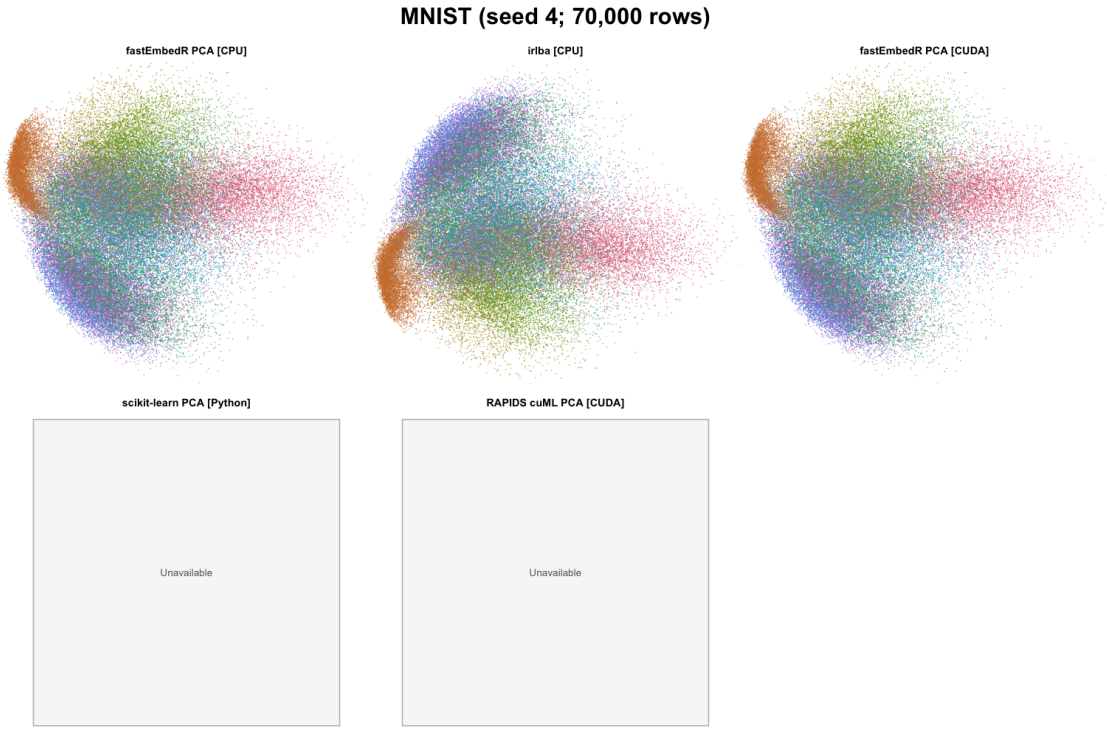

Figure S20: MNIST PCA outputs for the tested methods (seed 4; 70,000 rows). Colors encode benchmark labels and were not used for fitting.

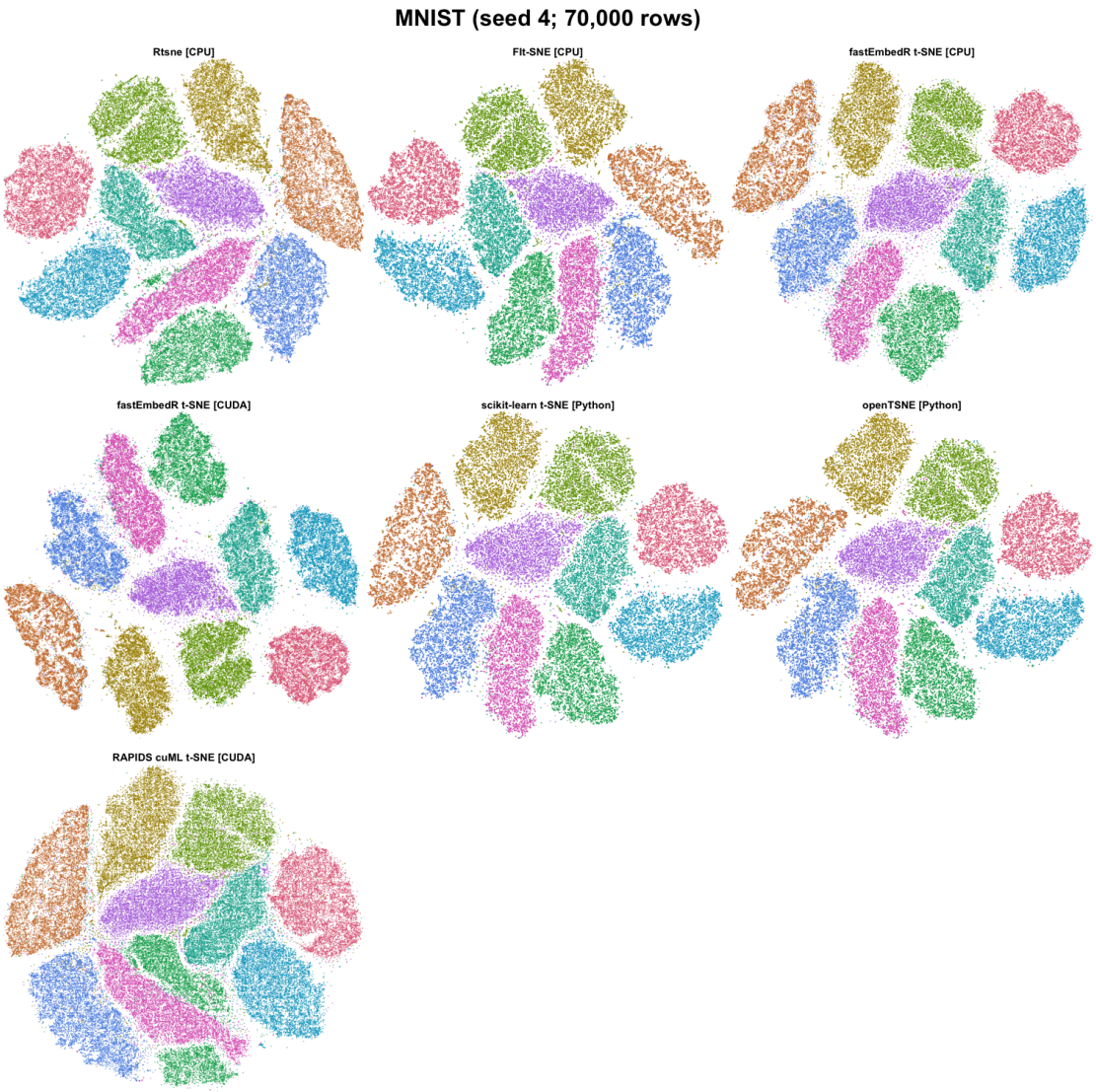

Figure S21: MNIST t-SNE outputs for the tested methods (seed 4; 70,000 rows). Colors encode benchmark labels and were not used for fitting.

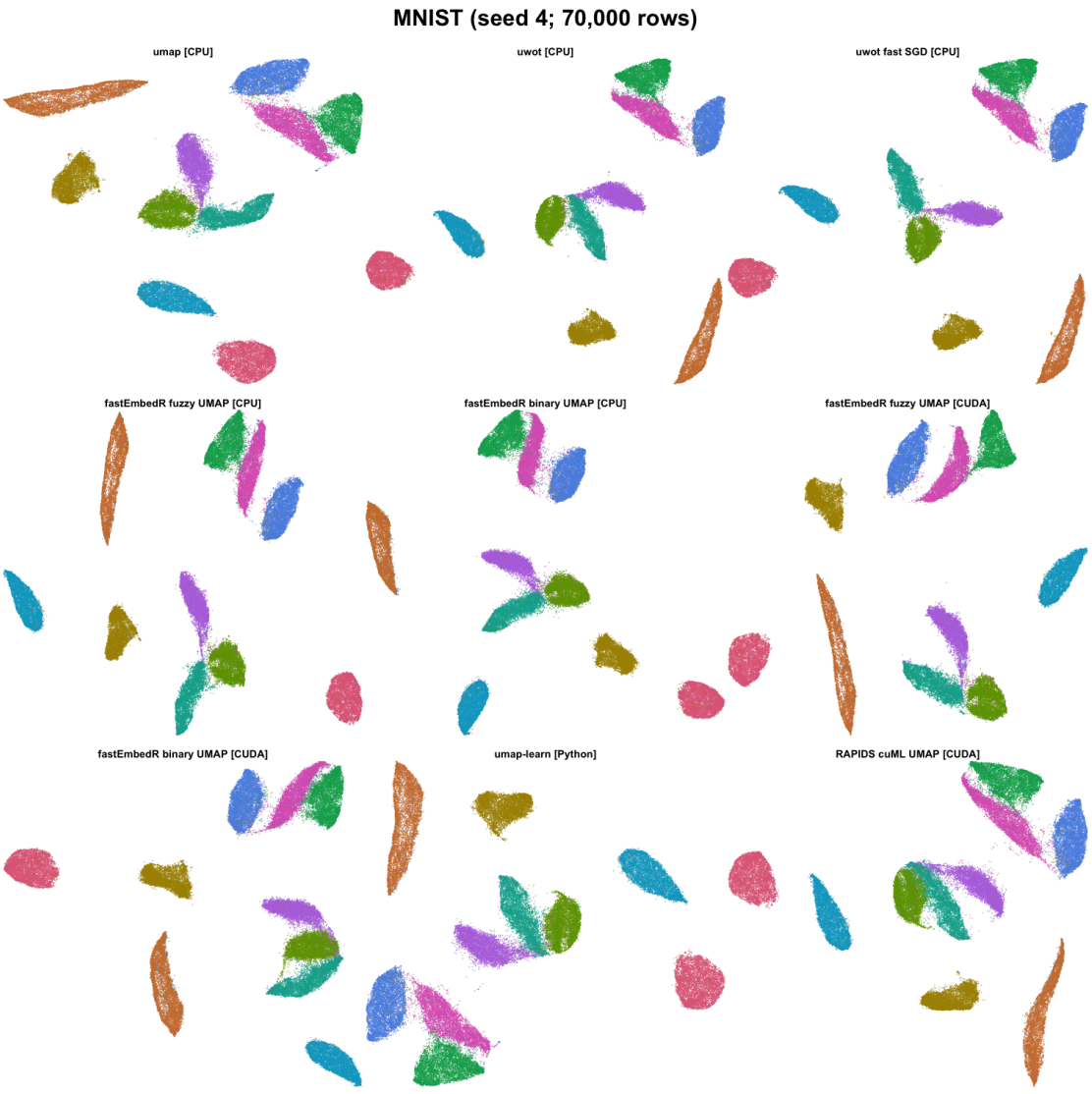

Figure S22: MNIST UMAP outputs for the tested methods (seed 4; 70,000 rows). Colors encode benchmark labels and were not used for fitting.

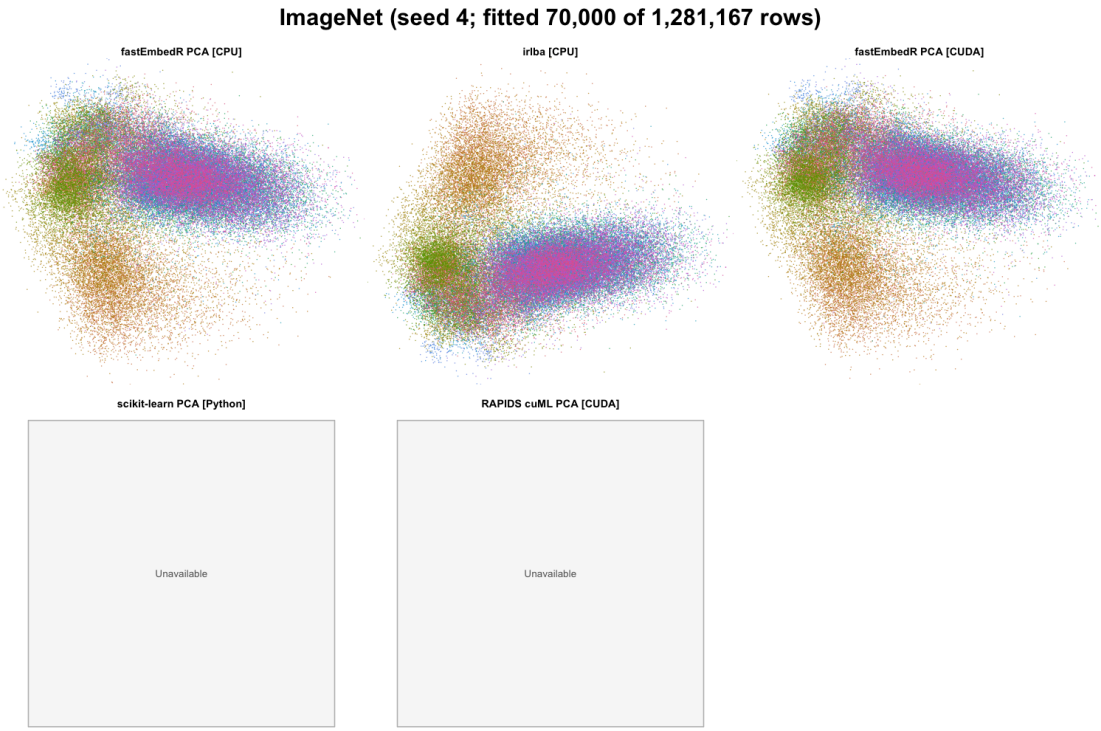

Figure S23: ImageNet PCA outputs for the tested methods (seed 4; fitted 70,000 of 1,281,167 rows). Colors encode benchmark labels and were not used for fitting.

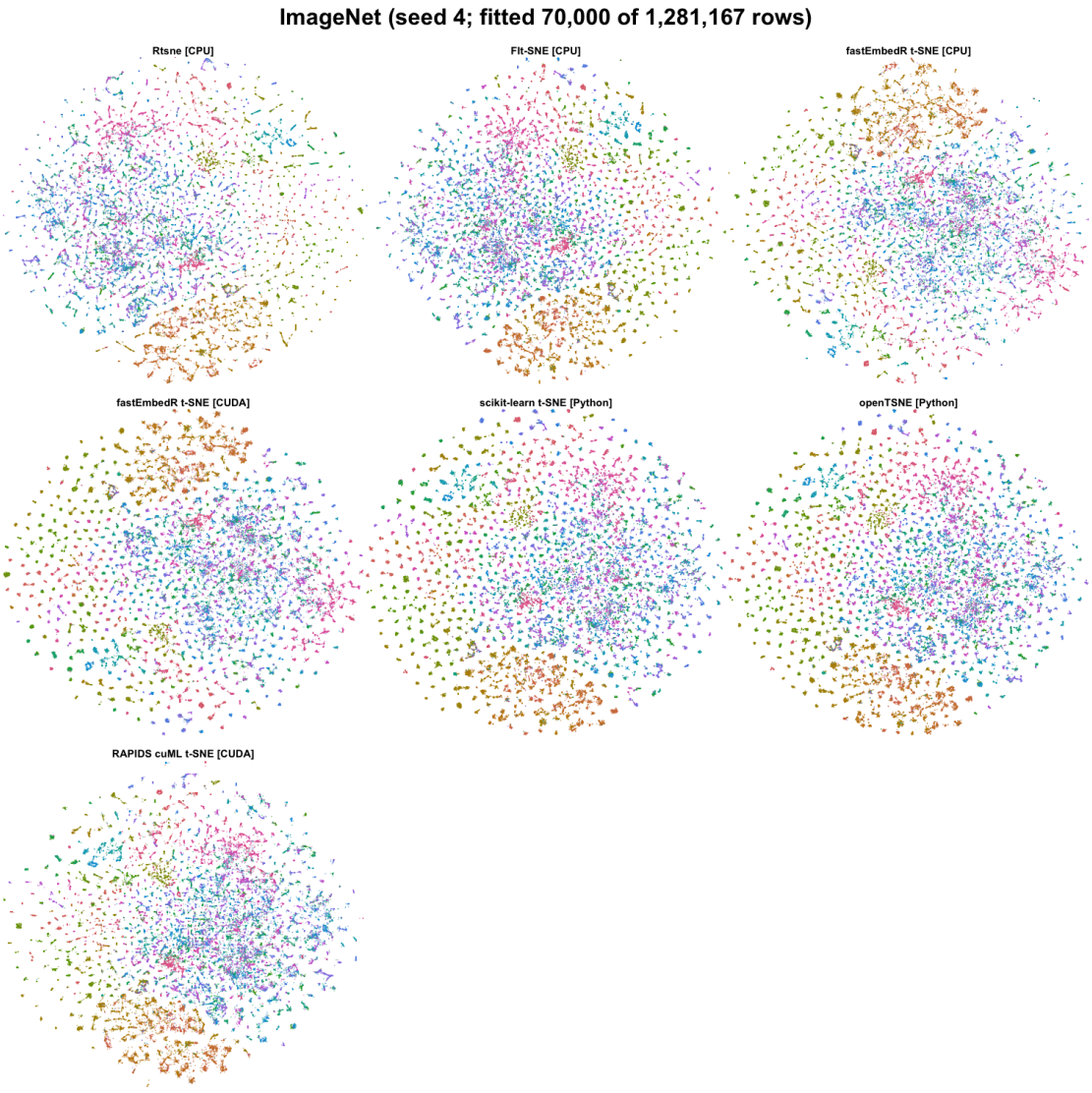

Figure S24: ImageNet t-SNE outputs for the tested methods (seed 4; fitted 70,000 of 1,281,167 rows). Colors encode benchmark labels and were not used for fitting.

Figure S25: ImageNet UMAP outputs for the tested methods (seed 4; fitted 70,000 of 1,281,167 rows). Colors encode benchmark labels and were not used for fitting.

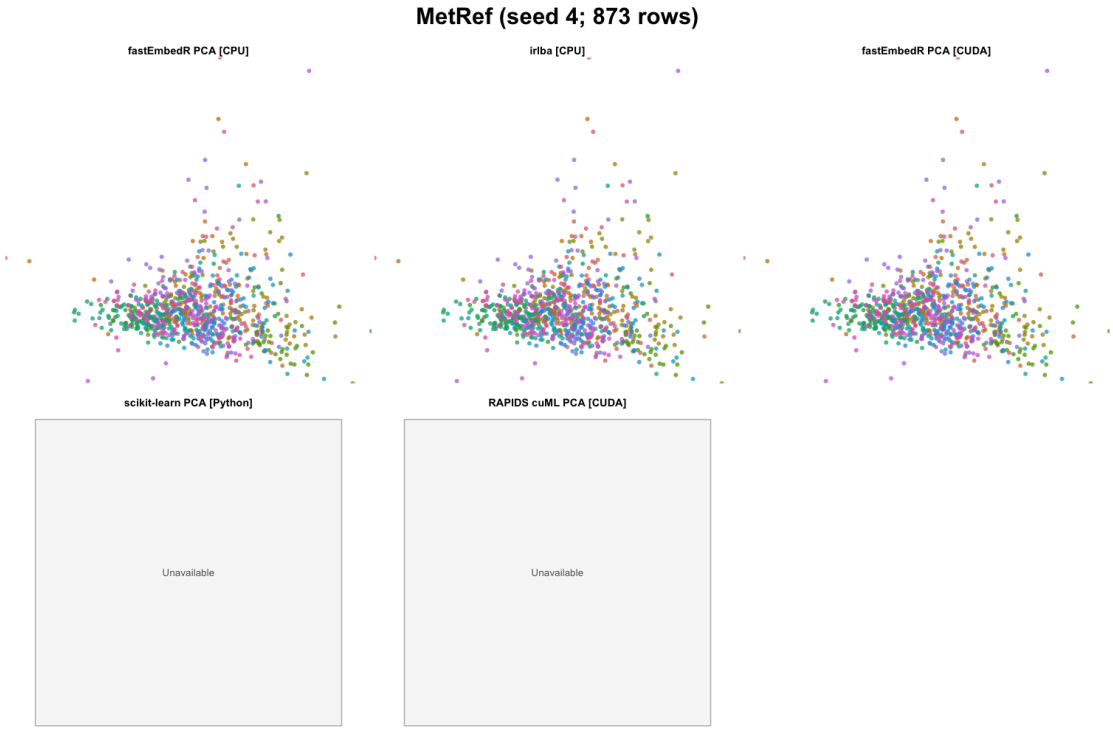

Figure S26: MetRef PCA outputs for the tested methods (seed 4; 873 rows). Colors encode benchmark labels and were not used for fitting.

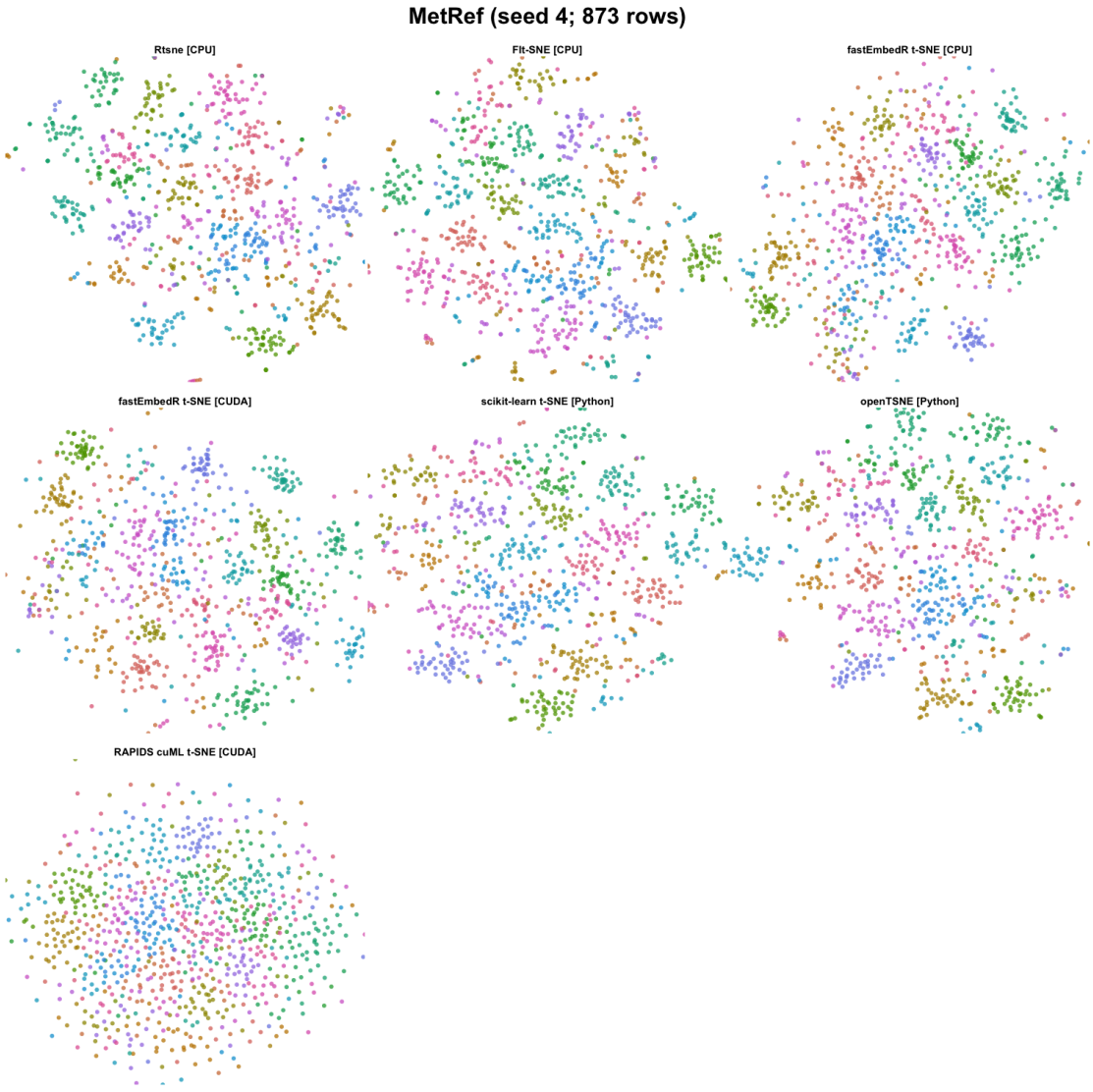

Figure S27: MetRef t-SNE outputs for the tested methods (seed 4; 873 rows). Colors encode benchmark labels and were not used for fitting.

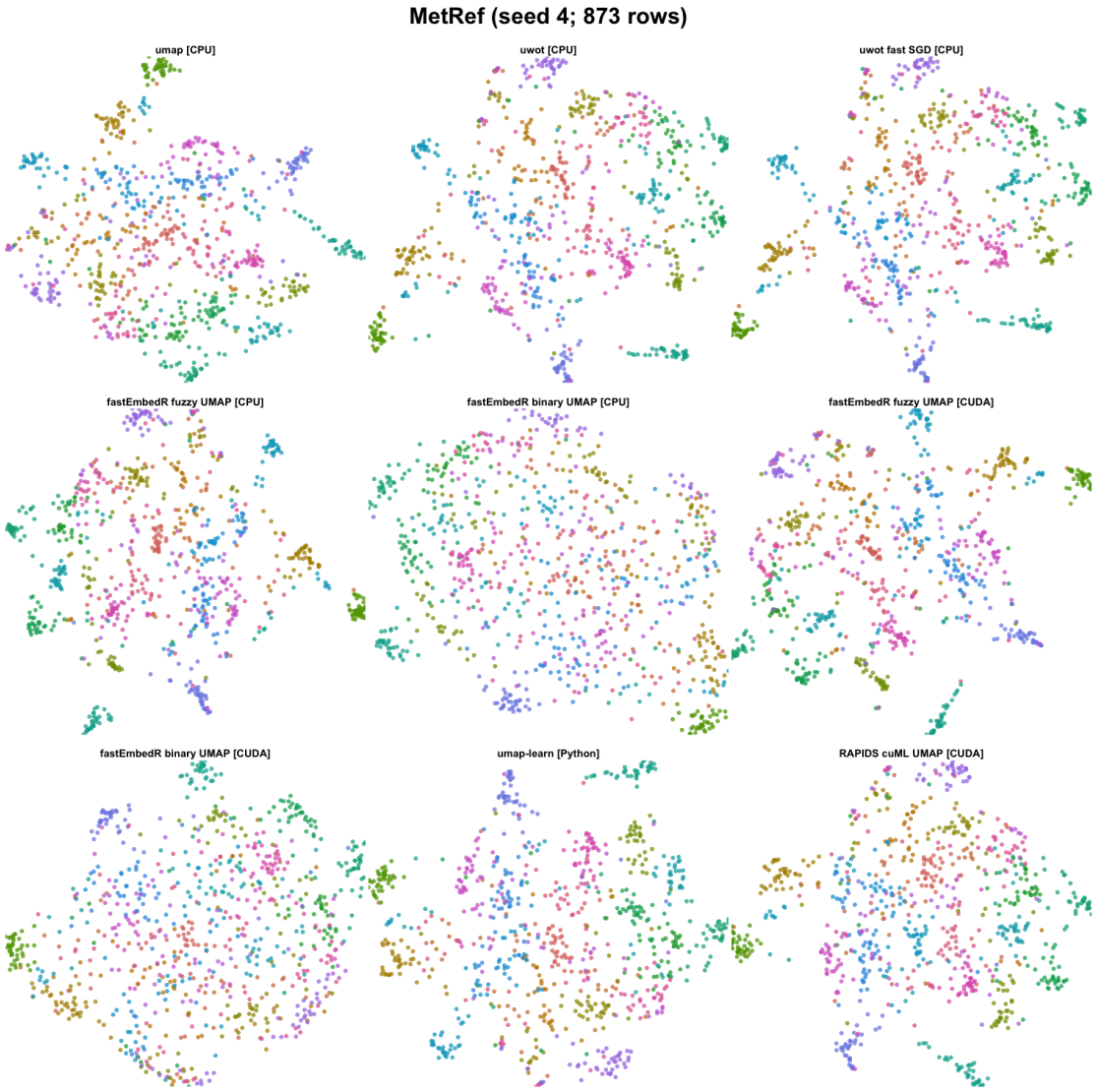

Figure S28: MetRef UMAP outputs for the tested methods (seed 4; 873 rows). Colors encode benchmark labels and were not used for fitting.

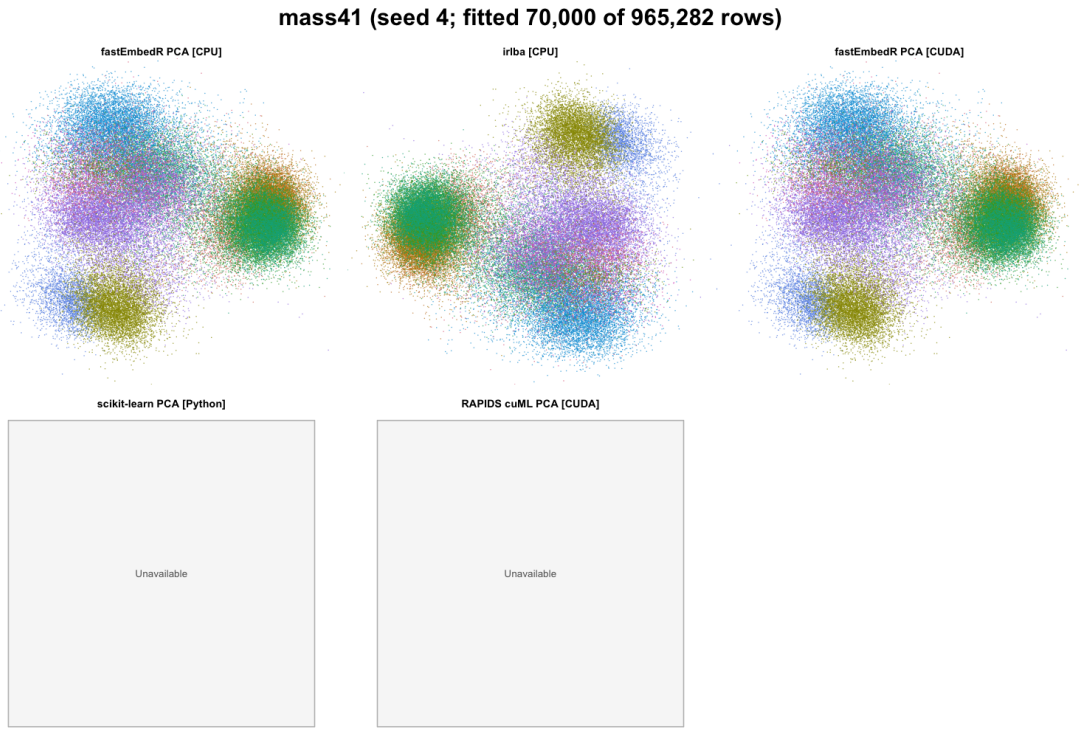

Figure S29: mass41 PCA outputs for the tested methods (seed 4; fitted 70,000 of 965,282 rows). Colors encode benchmark labels and were not used for fitting.

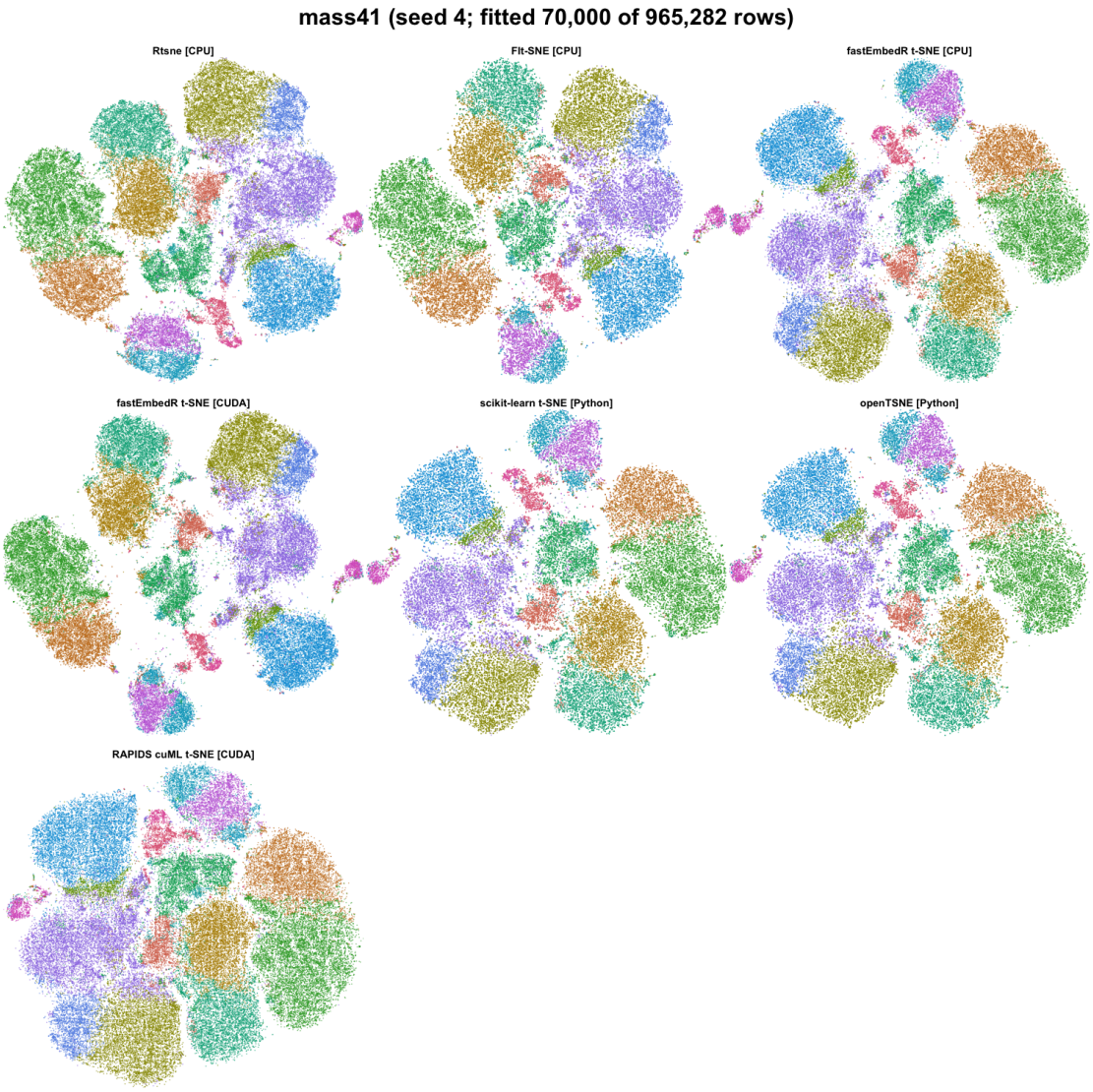

Figure S30: mass41 t-SNE outputs for the tested methods (seed 4; fitted 70,000 of 965,282 rows). Colors encode benchmark labels and were not used for fitting.

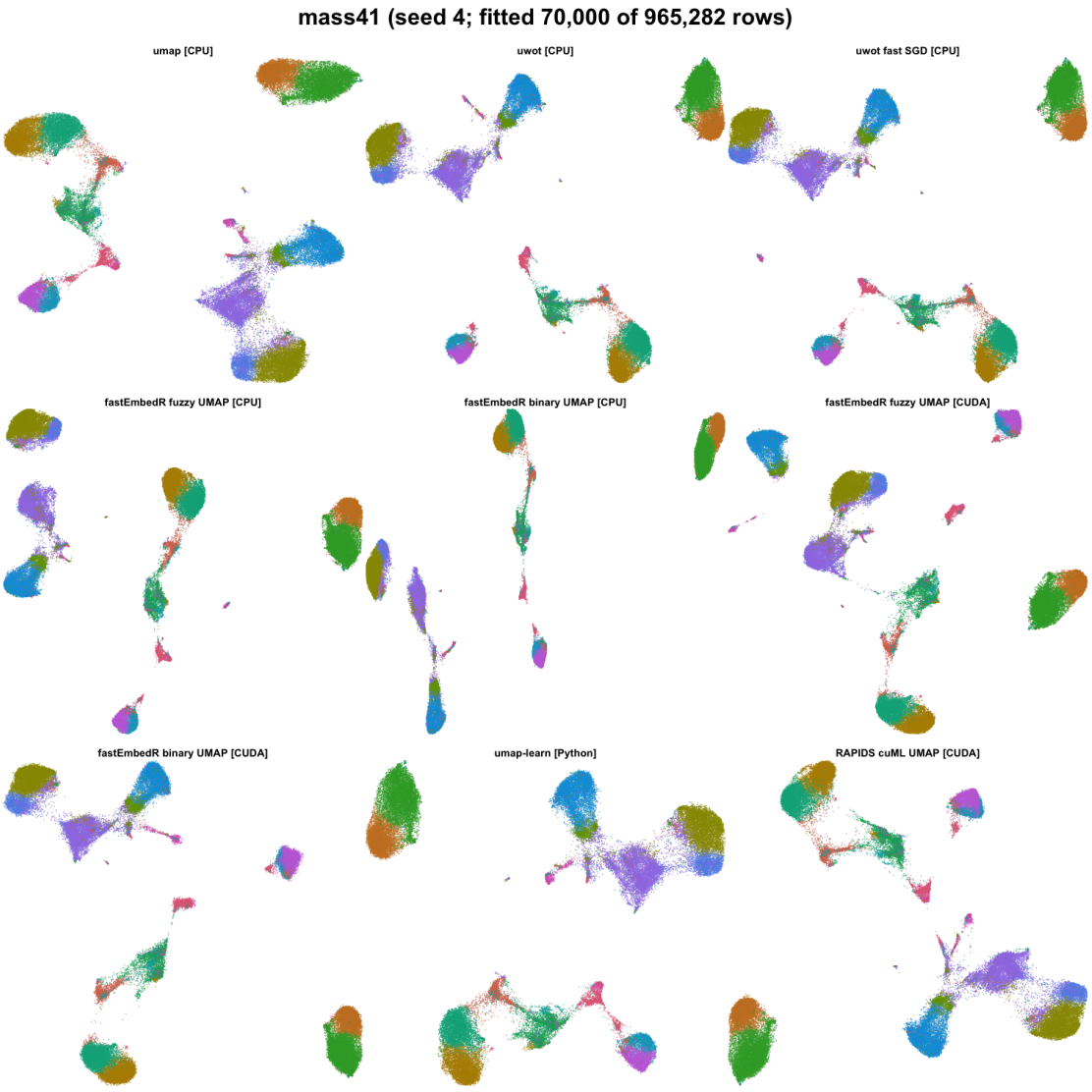

Figure S31: mass41 UMAP outputs for the tested methods (seed 4; fitted 70,000 of 965,282 rows). Colors encode benchmark labels and were not used for fitting.

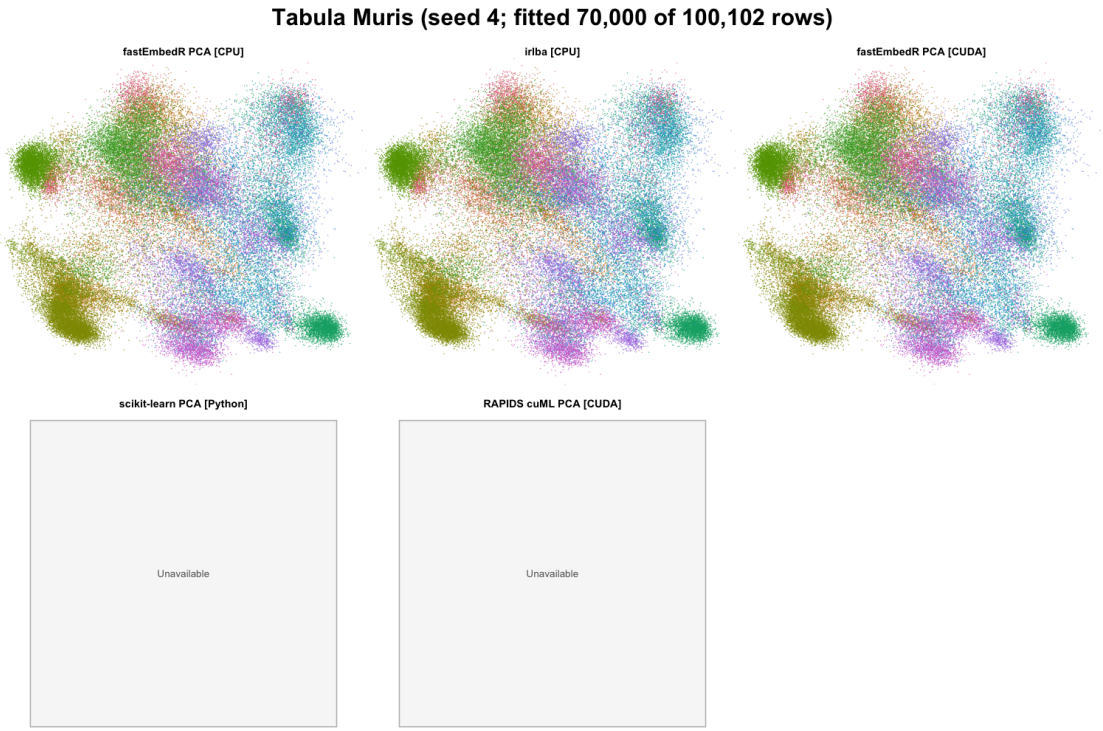

Figure S32: Tabula Muris PCA outputs for the tested methods (seed 4; fitted 70,000 of 100,102 rows). Colors encode benchmark labels and were not used for fitting.

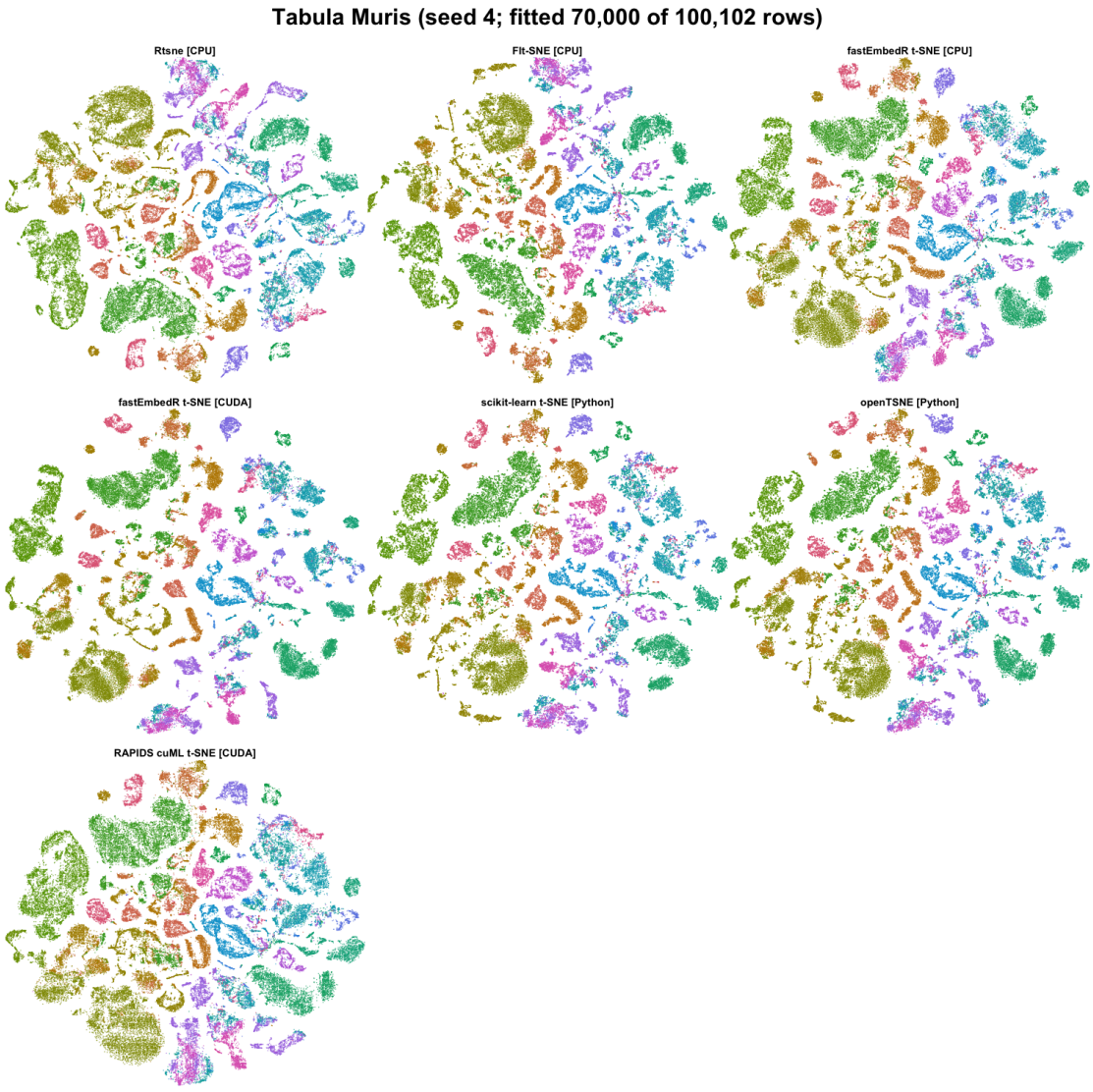

Figure S33: Tabula Muris t-SNE outputs for the tested methods (seed 4; fitted 70,000 of 100,102 rows). Colors encode benchmark labels and were not used for fitting.

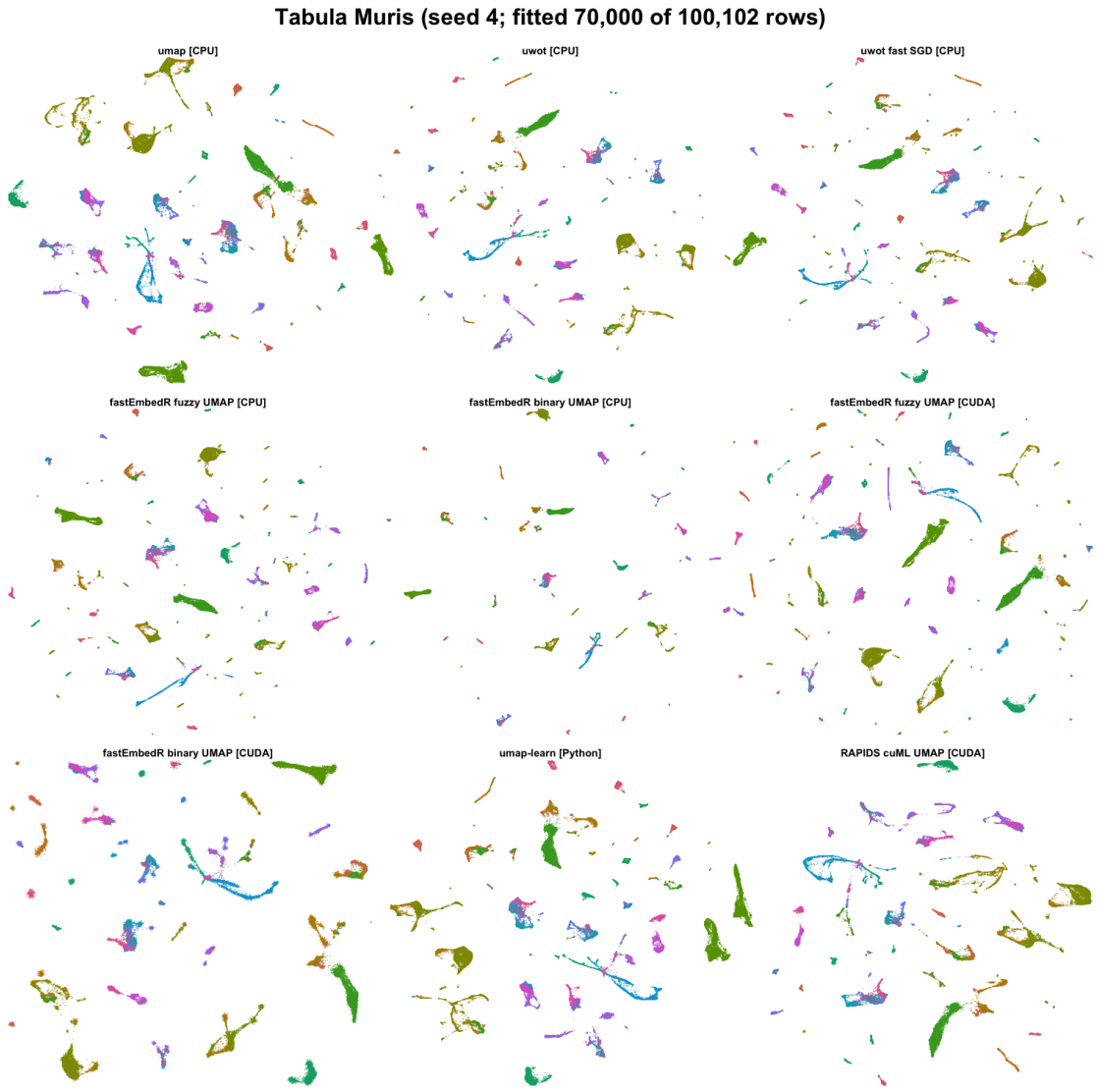

Figure S34: Tabula Muris UMAP outputs for the tested methods (seed 4; fitted 70,000 of 100,102 rows). Colors encode benchmark labels and were not used for fitting.

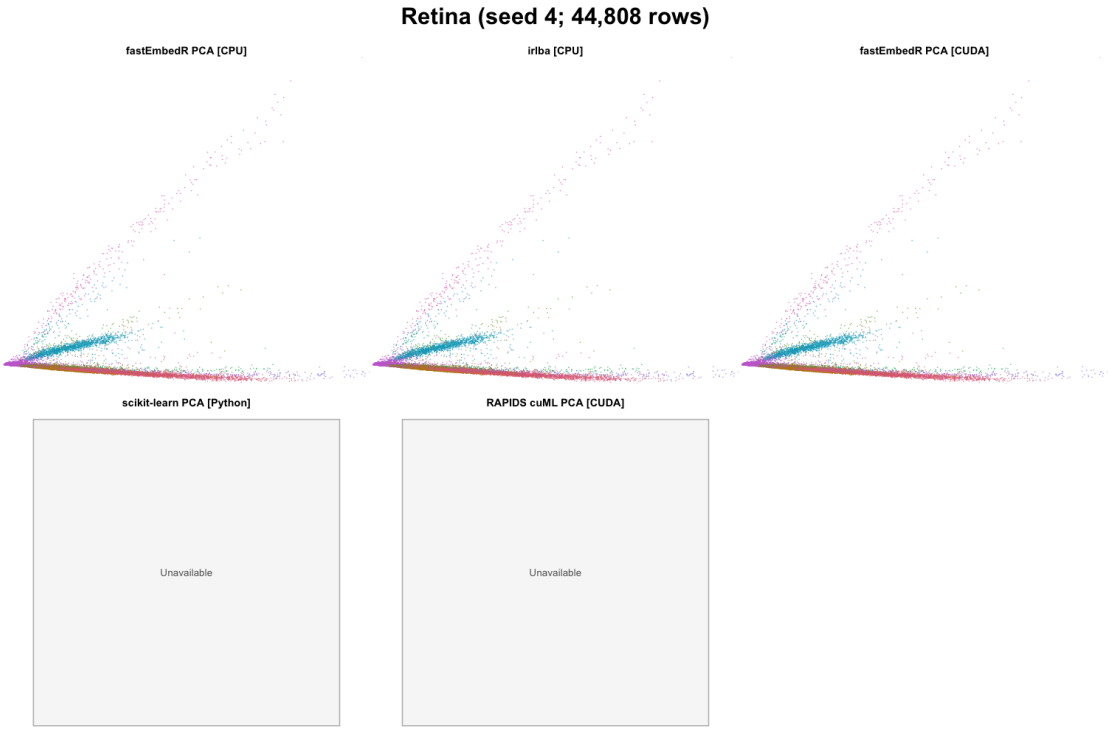

Figure S35: Retina PCA outputs for the tested methods (seed 4; 44,808 rows). Colors encode benchmark labels and were not used for fitting.

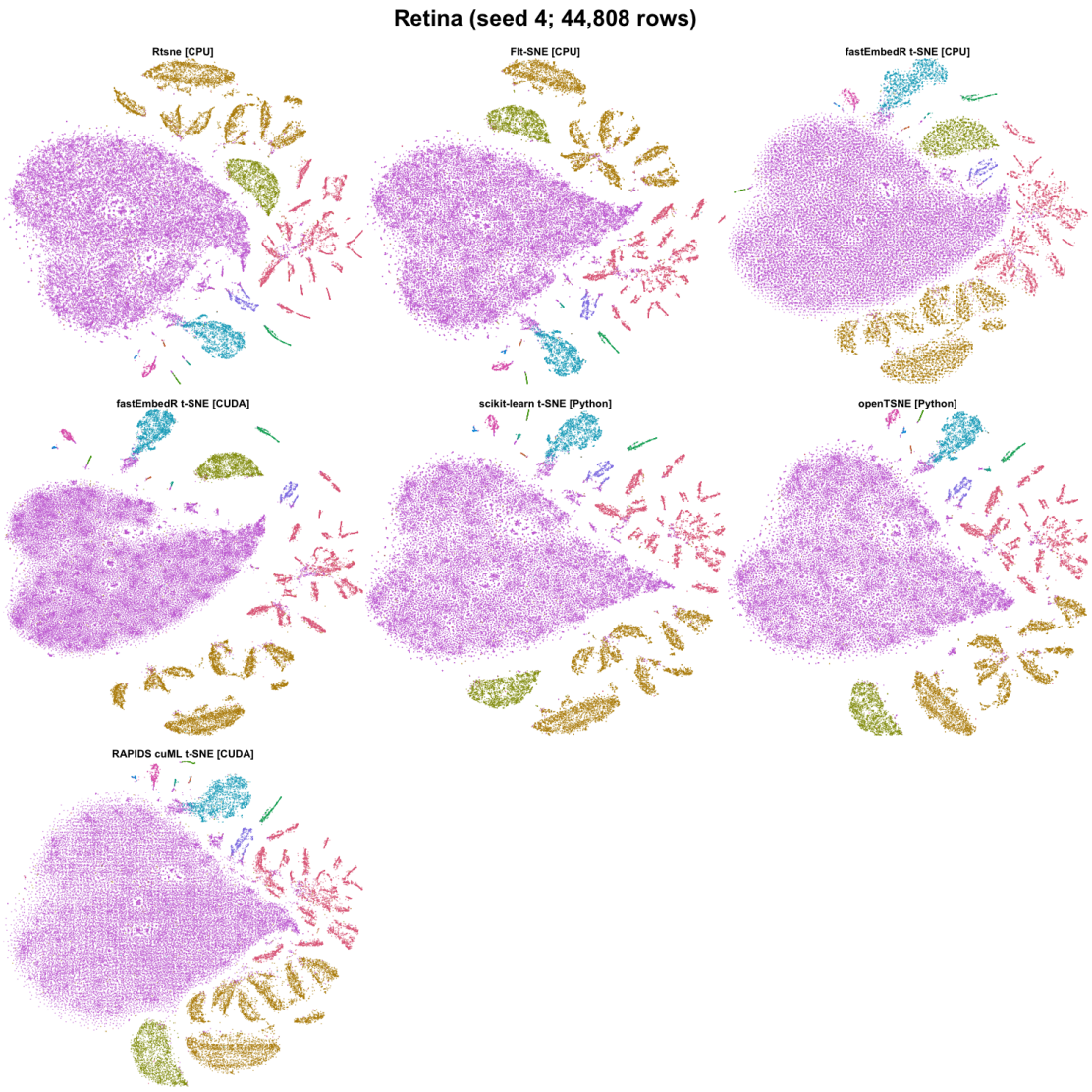

Figure S36: Retina t-SNE outputs for the tested methods (seed 4; 44,808 rows). Colors encode benchmark labels and were not used for fitting.

Figure S37: Retina UMAP outputs for the tested methods (seed 4; 44,808 rows). Colors encode benchmark labels and were not used for fitting.

11. Native library roles

Table S25 describes the current source interfaces, not the library versions or enabled features of the older diagnostic image. An optional build route is not evidence that its runtime or numerical quality was measured in that campaign. The package's own kernels implement the UMAP, t-SNE, graph, and clustering steps; none is delegated to cuML or cuGraph.

Table S25: Direct and transitive numerical-library interfaces. A requested unavailable accelerator route fails rather than switching to CPU.

Stage	Library or framework	Direct role and build condition
CPU numerical core	R BLAS, LAPACK, Fortran	Matrix products and small decompositions through R's configured toolchain.
CPU HNSW	**faissR**, FAISS	Versioned **faissR** C API builds and queries HNSW; FAISS is a dependency of **faissR**, not a direct **fastEmbedR** link. CPU exact search remains package-native.
Apple GPU	Metal, Metal Performance Shaders, MPS Graph, Accelerate, Foundation	Linked Apple SDK frameworks for native Metal kernels, transforms, and linear algebra; no CUDA libraries.
CUDA execution	CUDA runtime	Device allocation, transfers, native kernels, streams, and captured CUDA graphs.
CUDA neighbors	cuVS C API	Brute-force exact and IVF-Flat search when matrix-derived CUDA neighbors are requested. cuVS uses RAFT internally; this does not select fastEmbedR's direct RAFT PCA adapter.
CUDA PCA	cuBLAS, cuSOLVER, cuRAND	Native randomized-SVD products, factorizations, and sketches. Optional direct RAFT/RMM enables resident truncated SVD; it is not the file-backed PCA algorithm.
CUDA t-SNE	cuFFT	FFT repulsion and package-owned attraction and update kernels; cuVS is used only if neighbors must first be constructed.
Multi-GPU KNN/PCA	CUDA runtime, cuVS when searching	Separate device row partitions and a host-side merge or output stage; an allocation of two GPUs alone does not prove a speedup.
Multi-GPU UMAP/t-SNE	NCCL	Direct collectives coordinate a single embedding across devices in experimental kernels. RAFT communications and distributed cuFFT are not called by this route.
Diagnostics	**gpuinfo**	Suggested package for external inspection; not used as a production compute backend.

RAFT's single-node multi-GPU resources could manage device streams and memory for the existing cuVS exact/IVF KNN route. The current implementation instead stages bounded file-backed shards and partitions queries itself; the cuVS resource handle alone would not remove its disk and merge stages. For embedding optimization, RAFT's communication abstraction could replace some direct NCCL plumbing, but would not provide the UMAP updates or global t-SNE repulsion. A comparative adapter would need identical graph, initialization, precision, device count, stopping rule, and output audit, followed by per-device memory and end-to-end time measurements. Until then, direct NCCL is the described optimizer implementation and RAFT multi-GPU acceleration is not a result.

12. Graph-supported layout-guided cluster merging

The optional `layout_merge` control in `graph_cluster()` is an exploratory postprocessing rule for a resident graph, not a replacement for Leiden, Louvain, or Walktrap. PAGA motivates assessing connectivity between partitions on the original neighbor graph rather than inferring

it from the distance between two-dimensional clusters (Wolf, Hamey, Plass, Solana, Dahlin, G"ottgens, Rajewsky, Simon, and Theis 2019). For each pair of communities, our rule divides the number of positive-weight cross-edges by its expected count under random assignment of graph edge ends. It also divides the median plotted length of those cross-edges by the median plotted length of all within-community graph edges. A pair is eligible only when it has at least two cross-edges, connectivity at least 0.2, and length ratio at most 2. Eligible pairs are joined by connected components. These thresholds are explicit defaults, not a calibrated biological criterion; the rule and its layout condition are our adaptation, *not* the published PAGA estimator or a claim that visual proximity establishes a shared cell state. t-SNE geometry in particular depends on its visualization settings (Kobak and Berens 2019).

Figure S38 is a controlled 18-vertex synthetic example generated by `analysis/layout_merge_showcase.R`. The two adjacent six-vertex communities have three original-graph links and meet both gates (connectivity 0.258; length ratio 1.00); the third also has three links but fails the layout gate (ratio 28.78). The result contains the original and merged memberships, both modularity values, resolved thresholds, and a pair-level decision table. This is a mechanism demonstration, not evidence that clustering accuracy improves on the benchmark data sets; such a claim needs labels or independent structure and sensitivity analysis across UMAP/t-SNE seeds.

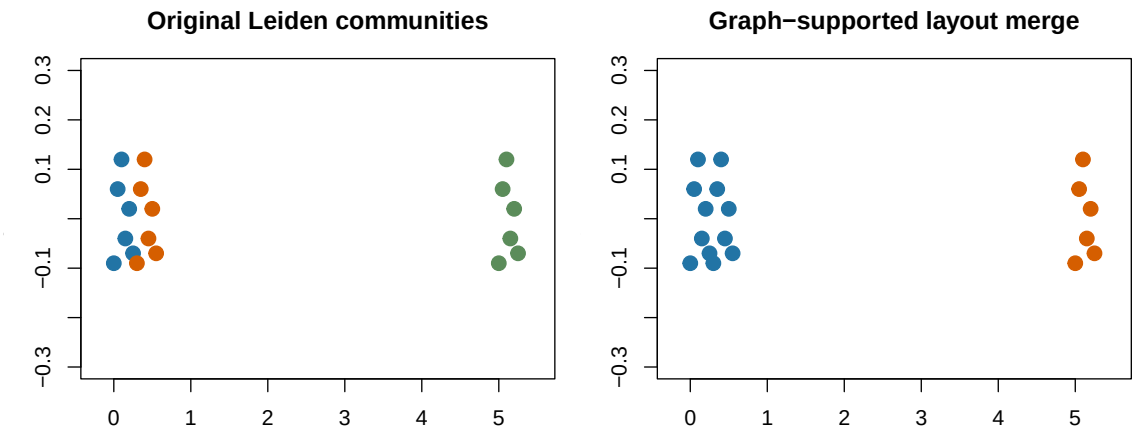

Figure S38: Original and layout-guided communities on the same synthetic graph. Colors are algorithmic community labels, not biological classes. The large empty space is an intentionally constructed negative control.

13. Principal workflow settings

Table S26 separates PCA used to initialize a layout from PCA feature preprocessing. “Policy matched” denotes an initialization class, not shared coordinates; the full-call timings include each implementation’s initialization. Additional seed, precision, optimizer, and thread settings remain in the machine-readable campaign manifest.

Table S26: Principal settings in the revised workflow protocol. This is the requested protocol, not a claim that every diagnostic fit completed.

Method	Init.	PCA features	Iterations/epochs	Support	Learning rate
fastEmbedR t-SNE	PCA	None	250 + 750	$k = P = 30$	$\max(n/12, 200)$
Rtsne	PCA	None	1,000	Internal $3P$	200
FIt-SNE	PCA	None	1,000	Internal $3P$	Automatic
openTSNE	PCA	None	250 + 750	Internal $3P$	Automatic
scikit-learn t-SNE	PCA	None	1,000	Internal $3P$	Automatic
cuML t-SNE	PCA	None	1,000	$3P$ requested	Capped $n/48$
fastEmbedR UMAP	Spectral	None	500, 300, or 200	$k = 30$	1 or 1.25
uwot	Spectral	None	Size-dependent	$k = 30$	1
umap	Spectral	None	200	$k = 30$	1
umap-learn	Spectral	None	500 or 200	$k = 30$	1
cuML UMAP	Spectral	None	500 or 200	$k = 30$	1

References

Kobak D, Berens P (2019). "The Art of Using t-SNE for Single-Cell Transcriptomics." *Nature Communications*, **10**, 5416. doi:10.1038/s41467-019-13056-x.

Wolf FA, Hamey FK, Plass M, Solana J, Dahlin JS, G"ottgens B, Rajewsky N, Simon L, Theis FJ (2019). "PAGA: Graph Abstraction Reconciles Clustering with Trajectory Inference through a Topology Preserving Map of Single Cells." *Genome Biology*, **20**, 59. doi:10.1186/s13059-019-1663-x.

Affiliation:

[1]Bioinformatics Unit, International Centre for Genetic Engineering and Biotechnology (ICGEB), Cape Town 7925, South Africa
[2]Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine, University of Cape Town, Cape Town 7925, South Africa
[3]Computational Biology Group, International Centre for Genetic Engineering and Biotechnology (ICGEB), Trieste, Italy
[4]Life Science Department (DSV), University of Trieste, Trieste, Italy
[5]Department of Chemistry "Ugo Schiff", University of Florence, Sesto Fiorentino, Italy
[6]Magnetic Resonance Center, University of Florence, Sesto Fiorentino, Italy
[7]Translational Bioinformatics Group, ICGEB, New Delhi, India
[8]Bioinformatics Facility, Department of Cellular, Computational and Integrative Biology - CIBIO, University of Trento, Trento, Italy
*These authors contributed equally.
†Co-corresponding authors: tenori@cerm.unifi.it and stefano.cacciatore@icgeb.org
URL: https://github.com/tkcaccia/fastEmbedR-extra

Journal of Statistical Software https://www.jstatsoft.org/
published by the Foundation for Open Access Statistics https://www.foastat.org/
MMMMMM YYYY, Volume VV, Issue II *Submitted:* yyyy-mm-dd
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd